Математическая логика и теория алгоритмов

Двоичные функции

применения

Применение двоичных функций:

проблема SAT (Satisfiability) – *проблема выполнимости булевой функции*



Типы булевых функций

- Типы БФ:

невыполнимые, выполнимые, тавтологии (общезначимые)

$x_1x_2x_3$	f1	f2	f3
0 0 0	0	0	1
0 0 1	0	0	1
0 1 0	0	1	1
0 1 1	0	0	1
1 0 0	0	0	1
1 0 1	0	0	1
1 1 0	0	1	1
1 1 1	0	1	1

НЕВЫПОЛНИМАЯ –
на всех значениях аргументов f равна 0

ВЫПОЛНИМАЯ –
есть значения аргументов, на которых f = 1

ТАВТОЛОГИЯ (общезначимая) –
на всех значениях аргументов f = 1

Проверка того, является ли функция выполнимой, является важнейшей проблемой Computer Science. Она называется SAT (Satisfiability Problem)



Семантика SAT

- Атомы: $P = \{p_1, p_2, \ldots p_n\}$
- Литералы: p_i и $\neg p_i$ для любых i
- Клоз c – множество литералов: $(\neg x \vee t \vee s)$ или {-x, t, s}
- Формула F – множество клозов
- Оценивание (valuation)
  - для атомов v: $P \to \{0,1\}$ (присваивание каждому атому 0 или 1}
  - клоз выполняется (v(c) =1), если для какого-нибудь литерала λ $v(\lambda)=1$
  - формула F выполняется, если ВСЕ ее клозы выполняются
- Проблема SAT:

Дана формула F как множество клозов.

Существует ли хотя бы одно оценивание (интерпретация), на котором F равна 1?

Если такие интерпретации существуют, то выдать их



Проблема выполнимости (SAT satisfiability)

- Проблема выполнимости – первая проблема, которая вошла в класс NP-полных проблем
- Хотя проблема SAT кажется тривиальной, известные алгоритмы ее решения являются экспоненциально сложными
- Например, проблема SAT для N=100 требует порядка 2^{100} операций. Это огромное число: $2^{10} = 1024 \approx 10^3$, поэтому $2^{100} \approx 10^{30}$ операций.
  Если задачу решать на процессоре со скоростью миллиард оп/сек, то для ее решения нам потребуется 30 миллионов лет
- Формула представляется в КНФ: $F=(x \vee y)\ (x \vee \neg z)\ (\neg x \vee \neg s)\ (\neg x \vee t \vee s)$
  Клозы часто записываются просто как множества литералов.
  Отрицательные литералы записываются с '-'.
  Например: F = { {x, y}, {x, -z}, {-x,s}, {-x, t, s}
- Если формула представляется в другом виде, то либо она переводится в КНФ, либо используются специальные приемы решения



Проблема SAT эквивалентна 3SAT

- Проблема 3SAT – проверка выполнимости КНФ формулы, которая включает в каждом дизъюнкте не более трех литералов
- Теорема. Проблема 3SAT эквивалентна прблеме SAT
- Доказательство. Любой дизъюнкт, содержащий $N \geq 3$ литералов, может быть сведен к конъюнкции дизъюнктов, содержащих ровно 3 литерала.
- Пример:
  Пусть $F = (a \vee b \vee c \vee d \vee e)$.
  Покажем, что выполнимость F эквивалентна выполнимости
  $\Phi = (a \vee b \vee x) \wedge (\neg x \vee c \vee y) \wedge (\neg y \vee d \vee e)$

Решение задач

Теорема Кука (1971):
"Проблема выполнимости булевой формулы в КНФ (SAT),
$F = D_1 \wedge D_2 \wedge \ldots \wedge D_n$, *является сложной (NP-трудной)"*

- Проверка того, что формула в КНФ является общезначимой, очень проста: $D_1 \wedge D_2 \wedge \ldots \wedge D_n = 1$ iff $\forall i \; .D_i = 1$, а каждый дизъюнкт $D_i = 1$ iff этот дизъюнкт содержит пару контрарных литер (т.е. переменную и ее отрицание)
- Проверка того, что формула представленная в ДНФ, является общезначимой, т.е. $K_1 \vee K_2 \vee \ldots \vee K_n = 1$, является сложной (NP-трудной)
- Проверка невыполнимости формулы, представленной в ДНФ является очень простой:
  $K_1 \vee K_2 \vee \ldots \vee K_n = 0$ iff каждый конъюнкт K_i равен 0. Конъюнкт равен 0 тогда и только тогда, когда он содержит пару контрарных литер



Использование отрицания

$x_1x_2x_3$	f1	f2	f3
0 0 0	0	0	1
0 0 1	0	0	1
0 1 0	0	1	1
0 1 1	0	0	1
1 0 0	0	0	1
1 0 1	0	0	1
1 1 0	0	1	1
1 1 1	0	1	1

Проверка общезначимости сводится к проверке невыполнимости:

F общезначима ≡ ¬F невыполнима

Например, функция f1 невыполнима (f1 ≡ **0**), функция f3 – общезначима (тавтология, f3 ≡ **1**)

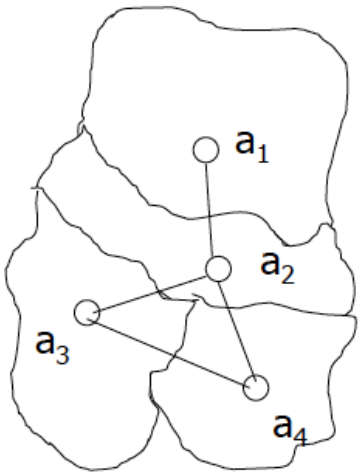
f1 = ¬f3; f3 = ¬f1

Решение проблемы невыполнимости решает и проблему выполнимости, и проблему общезначимости

Проблема раскраски карты

Множество проблем сводится к проблеме выполнимости

Карта



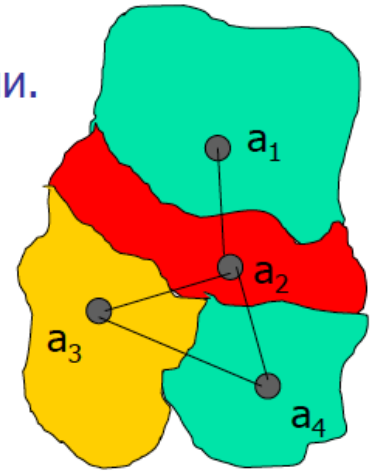
Раскрасить карту так, чтобы соседние страны были окрашены разными цветами.

Формализация.

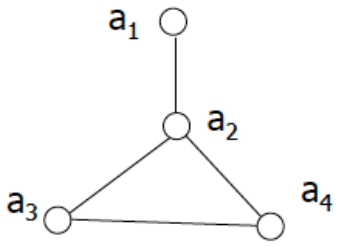
Даны граф G=(V, E), V={a_1, …, a_n}, и натуральное *k* (число различных цветов)

Существует ли такое назначение цветов вершинам графа, что у каждого ребра вершины разного цвета?

K-раскрашиваемый граф — граф, вершины которого можно раскрасить k цветами так, что у любого ребра концы будут разного цвета



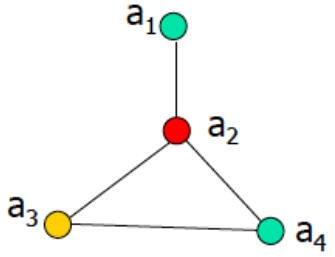
Граф



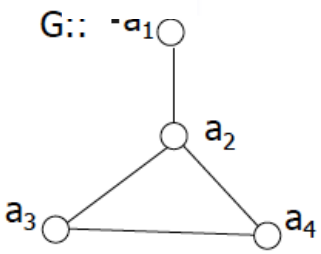
3 цвета:

C = 1 2 3

3-раскраска



Раскраска карты: четыре страны, три цвета. Сведение задачи к проблеме SAT



n=4 (четыре вершины) $(1 \le i \le n)$

k= 3 (три цвета) $(1 \le c \le k)$

C = 1 2 3

V={a_1, …, a_4}; E={(a_1, a_2), (a_2, a_4), (a_2, a_3), (a_3, a_4) } – четыре ребра

Двоичная переменная $x_{i,c}$ – вершина i окрашена цветом c

Всего 12 двоичных переменных

1. Каждая вершина окрашена каким-то цветом:

$$F1=(x_{1,1}\vee x_{1,2}\vee x_{1,3}) \wedge (x_{2,1}\vee x_{2,2}\vee x_{2,3}) \wedge (x_{3,1}\vee x_{3,2}\vee x_{3,3}) \wedge (x_{4,1}\vee x_{4,2}\vee x_{4,3})$$

2. Каждая вершина окрашена не более, чем одним цветом:

$$F2 = (x_{1,1} \Rightarrow \neg(x_{1,2} \vee x_{1,3})) \wedge (x_{1,2} \Rightarrow \neg(x_{1,1} \vee x_{1,3})) \wedge (x_{1,3} \Rightarrow \neg(x_{1,1} \vee x_{1,2})) \wedge \quad \text{// вершина 1}$$

$$\dots \quad \dots \quad \dots \qquad \qquad \qquad \qquad \qquad \qquad \qquad \wedge$$

$$(x_{4,1} \Rightarrow \neg(x_{4,2} \vee x_{4,3})) \wedge (x_{4,2} \Rightarrow \neg(x_{4,1} \vee x_{4,3})) \wedge (x_{4,3} \Rightarrow \neg(x_{4,1} \vee x_{4,2})) \wedge \quad \text{// вершина 4}$$

3. На каждом ребре цвета вершин различны:

$$F3= \neg(x_{1,1} \wedge x_{2,1}) \wedge \neg(x_{1,2} \wedge x_{2,2}) \wedge \neg(x_{1,3} \wedge x_{2,3}) \wedge \quad \text{// ребро } (a_1, a_2)$$

$$\dots \qquad \qquad \qquad \qquad \qquad \qquad \qquad \wedge$$

$$\neg(x_{3,1} \wedge x_{4,1}) \wedge \neg(x_{3,2} \wedge x_{4,2}) \wedge \neg(x_{3,3} \wedge x_{4,3}) \quad \text{// ребро } (a_3, a_4)$$

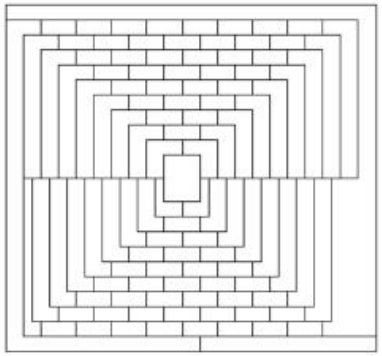
Проблема имеет решение, если функция $\Phi=F1 \wedge F2 \wedge F3$ выполнима

Проблема четырех красок

- С помощью алгоритма выполнимости (SAT) можно проверить, что для раскрашивания карты США достаточно 4-х красок.
- *Хватит ли четырех цветов для раскраски ЛЮБОЙ карты*? Это и есть знаменитая "Проблема 4-х красок"
- Проблема поставлена в 1852 г. Френсисом Гутри – он раскрашивал карту графств Англии и ему хватило 4-х красок.
- В 1879 г. Альфред Кемпе опубликовал доказательство в "American J. of Math". За "*крупный вклад в математику*" он тут же был избран членом Британского Королевского общества и возведен в рыцари. В 1890 г. в доказательстве Кемпе была найдена принципиальная ошибка.
- Теорема имеет долгую историю. Была доказана теорема о 5 красках, но теорему о 4-х красках доказать не удавалось.
- 1 апреля 1975 г. Мартин Гарднер опубликовал карту, для которой, как он предположил, нужно не меньше 5 красок.
- В 1976 году К.Аппелем и У.Хакеном из Иллинойского Ун-та теорема о 4-х красках была доказана с помощью компьютера.







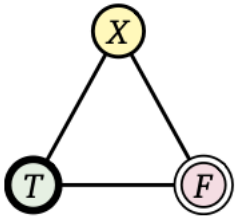
Карта Мартина Гарднера

Раскраска графа и 3SAT

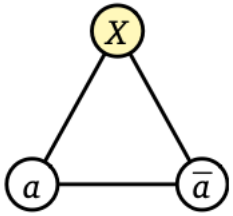
$$G = (V, E)$$

$$C : V \to \{1, 2, \ldots, k\}$$

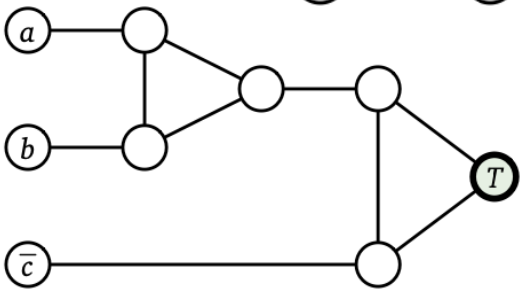
- *Имеется ли корректная раскраска в 3 цвета?*
- Три типа «гаджетов»:
  - гаджет истинности (truth gadget),



  - гаджет переменной (variable gadget),

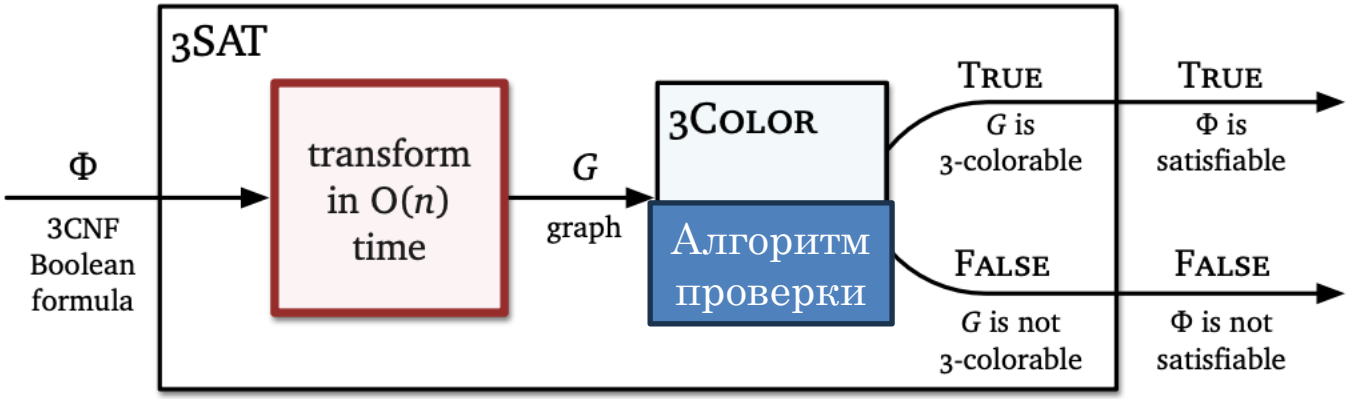


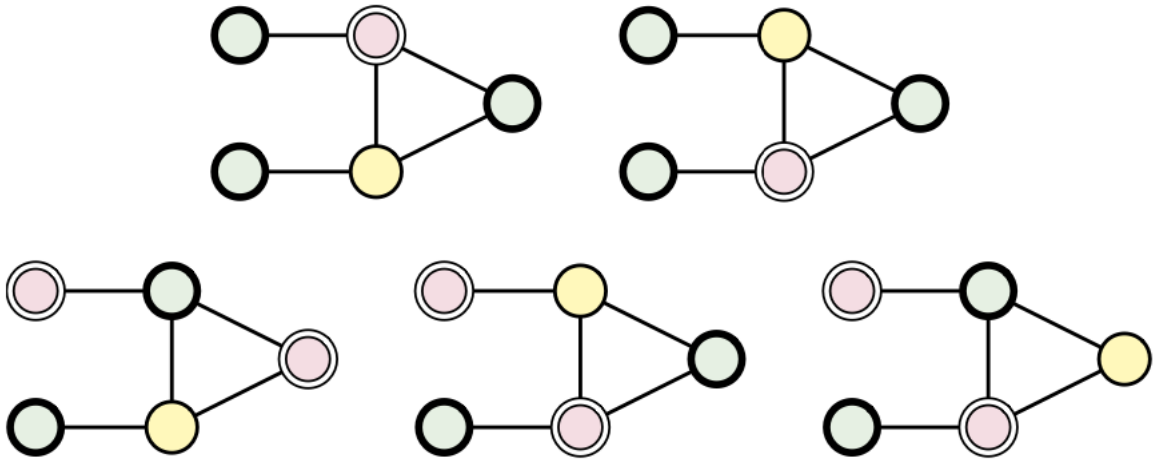
  - гаджет компоненты (clause gadget).



- **Каждый треугольник – как «мажоритарный логический элемент».**

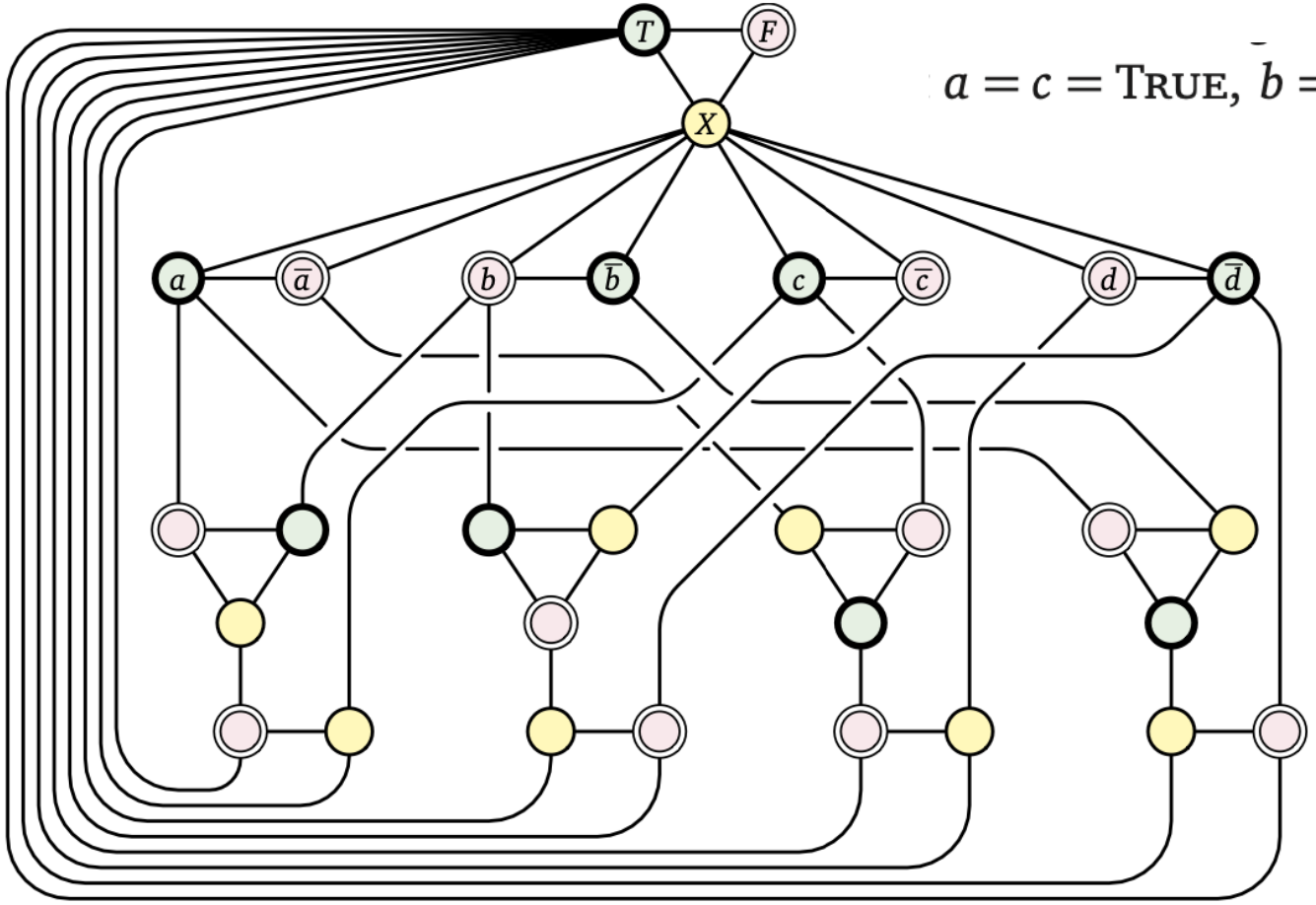
Раскраска графа и 3SAT





Все корректные раскраски в три цвета

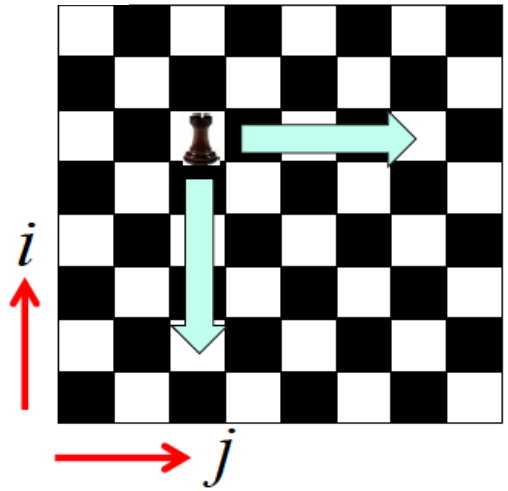
Раскраска графа и 3SAT



$a = c = \text{True}, \bar{b} = d = \text{False}.$

$$(a \vee b \vee c) \wedge (b \vee \bar{c} \vee \bar{d}) \wedge (\bar{a} \vee c \vee d) \wedge (a \vee \bar{b} \vee \bar{d})$$

Логические головоломки – сведение к SAT



- Проблема расстановки ладей
- Расставить на шахматной доске ладьи так, чтобы они не били друг друга

$x_{i,j}$ – двоичная переменная

$x_{i,j}$ = 1, если в поле (i, j) стоит ладья

Всего 64 двоичных переменных

$F=(\bigwedge_i \bigvee_j x_{i,j})$ // хотя бы одна ладья стоит на каждой горизонтали

$\wedge \bigwedge_{i,j} (x_{i,j} \Rightarrow \bigwedge_{i,\, s:\, s\neq j} \neg x_{i,\, s})$ // не более одной ладьи на горизонтали

$\wedge \bigwedge_{i,j} (x_{i,j} \Rightarrow \bigwedge_{k,\, j:\, k\neq i} \neg x_{k,\, j})$ // не более одной ладьи на вертикали

Любой набор значений аргументов, на котором F равна 1, дает решение задачи

Сведение проблемы эквивалентности к проблеме выполнимости

Вопрос: являются ли две заданные двоичные функции равносильными (эквивалентными)?

Пример: эквивалентны ли функции f и g?

$$f = \neg x_1 \neg x_2 \vee x_2 \neg x_3$$

$$g = \neg x_1 \vee x_2 \oplus x_3 x_2 (x_1 \vee x_3)$$

Теорема. Функции f и g эквивалентны тогда и только тогда, когда функция $f \oplus g$ невыполнима

- Доказательство. Самостоятельная работа

Проверка эквивалентности (равнозначности) двоичных функций сводится к проверке выполнимости



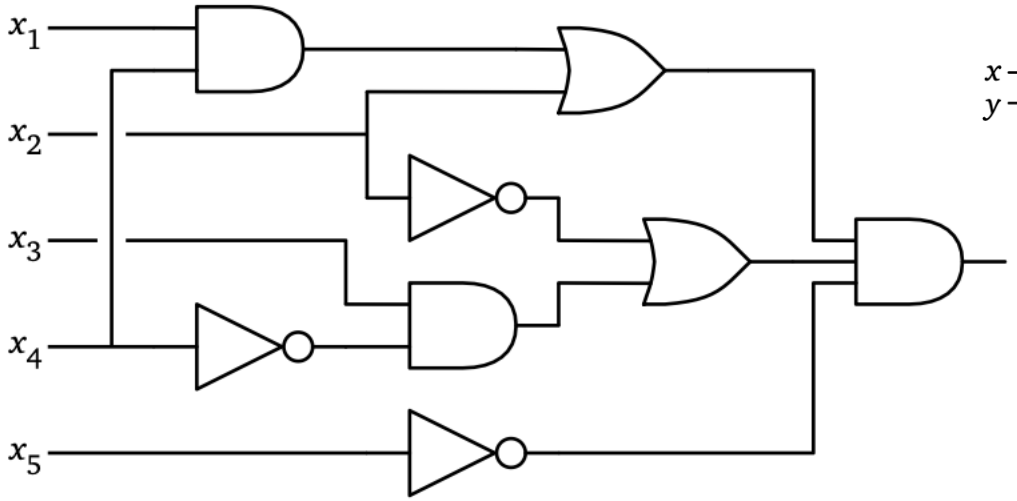
Проверка выполнимости логической схемы

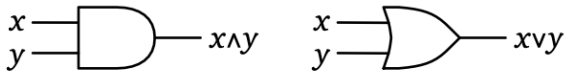
?

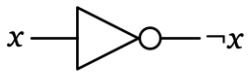
$$(a \vee b \vee c \vee \bar{d}) \Leftrightarrow ((b \wedge \bar{c}) \vee \overline{(\bar{a} \Rightarrow d)} \vee (c \neq a \wedge b)),$$

- Исходная схема К выполнима, если и только если полученная в итоге формула Ф выполнима. Доказать *необходимость* и *достаточность*.
- → Если задано множество входных данных, выполняющих логическую схему К, то можно вывести выполнимое присваивание для формулы Ф, вычислив выход каждого логического вентиля в схеме К.
- → Если задано выполнимое присваивание для формулы Ф, то можно получить выполнимые входные данные для логической схемы К, игнорируя переменные внутренних проводов и выходную переменную.

- *Преобразования из логической схемы к логической формуле можно выполнить за линейное время.*



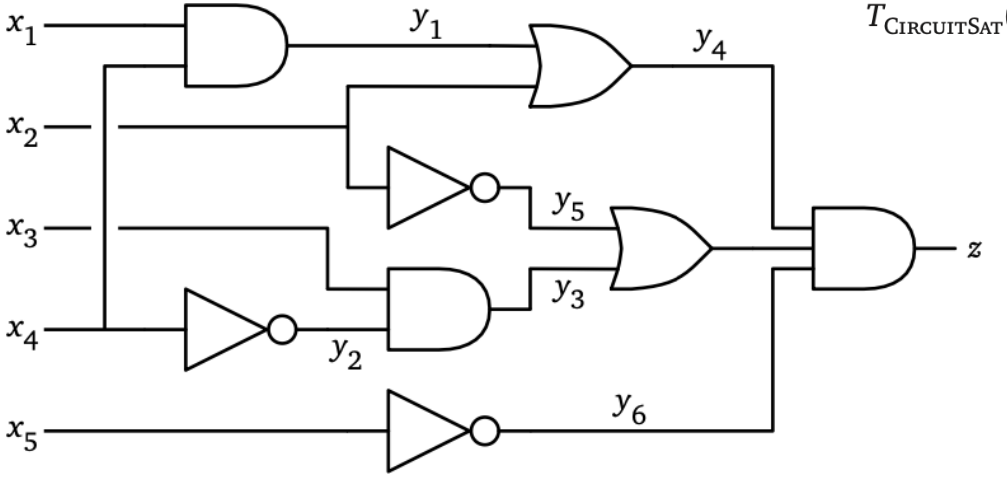




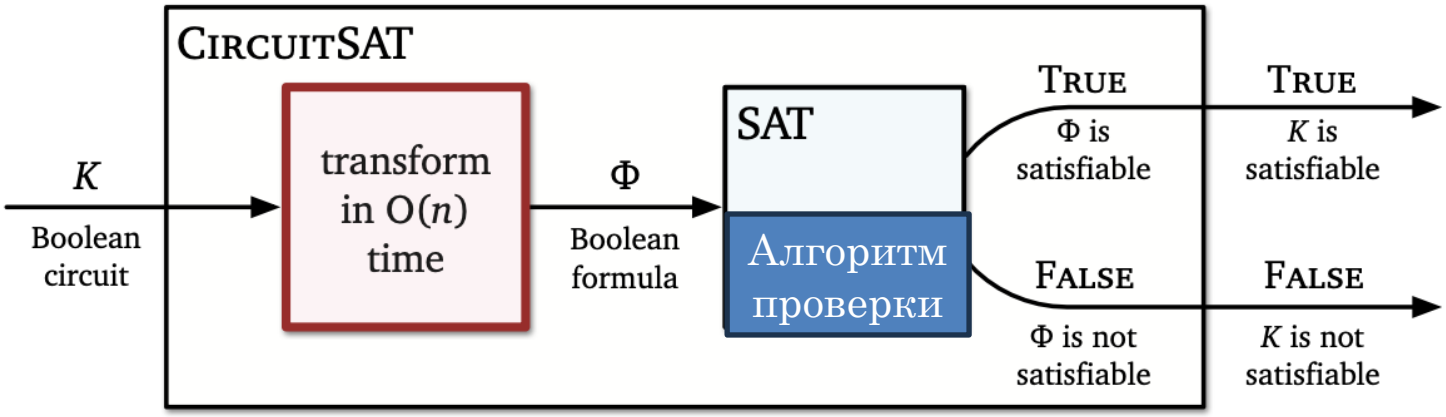
Проверка выполнимости логической схемы

CircuitSat(K):
transcribe K into a boolean formula Φ
return Sat(Φ) ⟪MAGIC⟫

$$T_{\text{CircuitSat}}(n) \le O(n) + T_{\text{SAT}}(O(n))$$

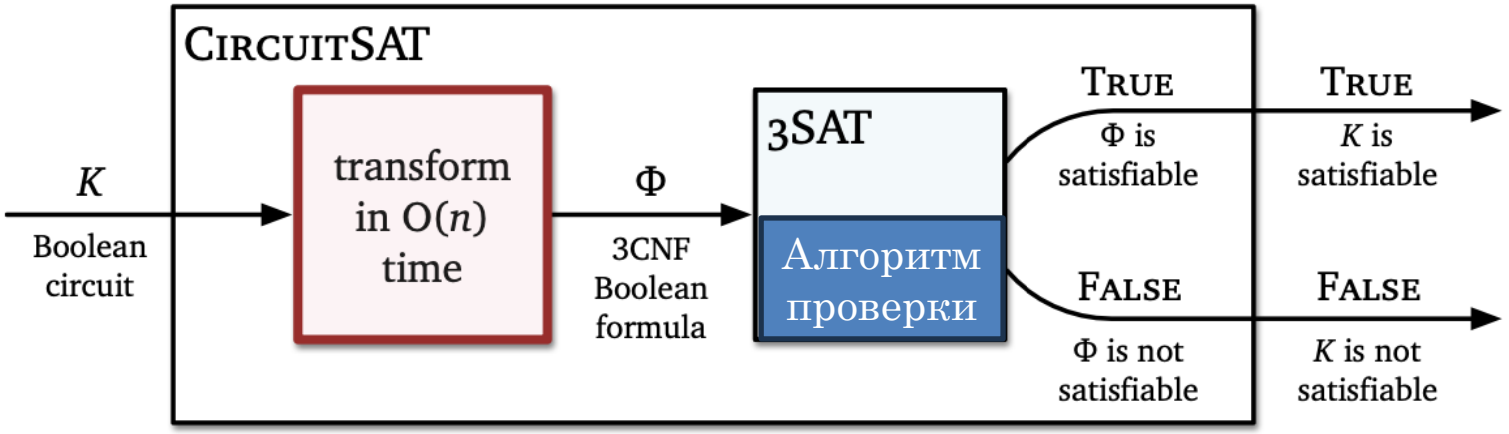


$$(y_1 = x_1 \wedge x_4) \wedge (y_2 = \overline{x_4}) \wedge (y_3 = x_3 \wedge y_2) \wedge (y_4 = y_1 \vee x_2) \wedge$$
$$(y_5 = \overline{x_2}) \wedge (y_6 = \overline{x_5}) \wedge (y_7 = y_3 \vee y_5) \wedge (z = y_4 \wedge y_7 \wedge y_6) \wedge z$$



3SAT
И ЛОГИЧЕСКАЯ СХЕМА

$$\overbrace{(a \vee b \vee c \vee d)}^{\text{clause}} \wedge (b \vee \bar{c} \vee \bar{d}) \wedge (\bar{a} \vee c \vee d) \wedge (a \vee \bar{b})$$



- *Make sure every and and or gate in K has exactly two inputs.* If any gate has $k > 2$ inputs, replace it with a binary tree of $k - 1$ binary gates.
- *Transcribe K' into a boolean formula* Φ1 *with one clause per gate,* exactly as in our previous reduction to Sat.
- *Replace each clause in* Φ1 *with a CNF formula.* There are only three types of clauses in Φ1, one for each type of gate in *K'*.
- *Replace each clause in* Φ2 *with a 3CNF formula.* Every clause in Φ2 has *at most* three literals, but we need clauses with *exactly* three literals.

Примеры вычислений

$$a = b \wedge c \quad \longmapsto \quad (a \vee \bar{b} \vee \bar{c}) \wedge (\bar{a} \vee b) \wedge (\bar{a} \vee c)$$
$$a = b \vee c \quad \longmapsto \quad (\bar{a} \vee b \vee c) \wedge (a \vee \bar{b}) \wedge (a \vee \bar{c})$$
$$a = \bar{b} \quad \longmapsto \quad (a \vee b) \wedge (\bar{a} \vee \bar{b})$$

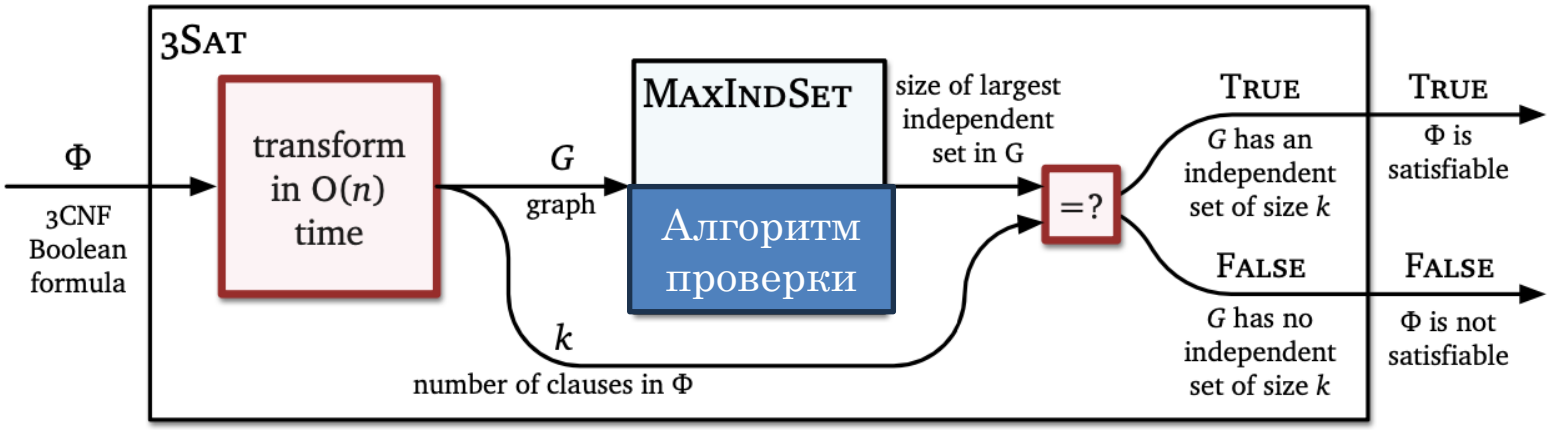
$$a \vee b \longmapsto (a \vee b \vee x) \wedge (a \vee b \vee \bar{x})$$
$$z \longmapsto (z \vee x \vee y) \wedge (z \vee \bar{x} \vee y) \wedge (z \vee x \vee \bar{y}) \wedge (z \vee \bar{x} \vee \bar{y})$$

$$(y_1 \vee \overline{x_1} \vee \overline{x_4}) \wedge (\overline{y_1} \vee x_1 \vee z_1) \wedge (\overline{y_1} \vee x_1 \vee \overline{z_1}) \wedge (\overline{y_1} \vee x_4 \vee z_2) \wedge (\overline{y_1} \vee x_4 \vee \overline{z_2})$$
$$\wedge (y_2 \vee x_4 \vee z_3) \wedge (y_2 \vee x_4 \vee \overline{z_3}) \wedge (\overline{y_2} \vee \overline{x_4} \vee z_4) \wedge (\overline{y_2} \vee \overline{x_4} \vee \overline{z_4})$$
$$\wedge (y_3 \vee \overline{x_3} \vee \overline{y_2}) \wedge (\overline{y_3} \vee x_3 \vee z_5) \wedge (\overline{y_3} \vee x_3 \vee \overline{z_5}) \wedge (\overline{y_3} \vee y_2 \vee z_6) \wedge (\overline{y_3} \vee y_2 \vee \overline{z_6})$$
$$\wedge (\overline{y_4} \vee y_1 \vee x_2) \wedge (y_4 \vee \overline{x_2} \vee z_7) \wedge (y_4 \vee \overline{x_2} \vee \overline{z_7}) \wedge (y_4 \vee \overline{y_1} \vee z_8) \wedge (y_4 \vee \overline{y_1} \vee \overline{z_8})$$
$$\wedge (y_5 \vee x_2 \vee z_9) \wedge (y_5 \vee x_2 \vee \overline{z_9}) \wedge (\overline{y_5} \vee \overline{x_2} \vee z_{10}) \wedge (\overline{y_5} \vee \overline{x_2} \vee \overline{z_{10}})$$
$$\wedge (y_6 \vee x_5 \vee z_{11}) \wedge (y_6 \vee x_5 \vee \overline{z_{11}}) \wedge (\overline{y_6} \vee \overline{x_5} \vee z_{12}) \wedge (\overline{y_6} \vee \overline{x_5} \vee \overline{z_{12}})$$
$$\wedge (\overline{y_7} \vee y_3 \vee y_5) \wedge (y_7 \vee \overline{y_3} \vee z_{13}) \wedge (y_7 \vee \overline{y_3} \vee \overline{z_{13}}) \wedge (y_7 \vee \overline{y_5} \vee z_{14}) \wedge (y_7 \vee \overline{y_5} \vee \overline{z_{14}})$$
$$\wedge (y_8 \vee \overline{y_4} \vee \overline{y_7}) \wedge (\overline{y_8} \vee y_4 \vee z_{15}) \wedge (\overline{y_8} \vee y_4 \vee \overline{z_{15}}) \wedge (\overline{y_8} \vee y_7 \vee z_{16}) \wedge (\overline{y_8} \vee y_7 \vee \overline{z_{16}})$$
$$\wedge (y_9 \vee \overline{y_8} \vee \overline{y_6}) \wedge (\overline{y_9} \vee y_8 \vee z_{17}) \wedge (\overline{y_9} \vee y_6 \vee z_{18}) \wedge (\overline{y_9} \vee y_6 \vee \overline{z_{18}}) \wedge (\overline{y_9} \vee y_8 \vee \overline{z_{17}})$$
$$\wedge (y_9 \vee z_{19} \vee z_{20}) \wedge (y_9 \vee \overline{z_{19}} \vee z_{20}) \wedge (y_9 \vee z_{19} \vee \overline{z_{20}}) \wedge (y_9 \vee \overline{z_{19}} \vee \overline{z_{20}})$$

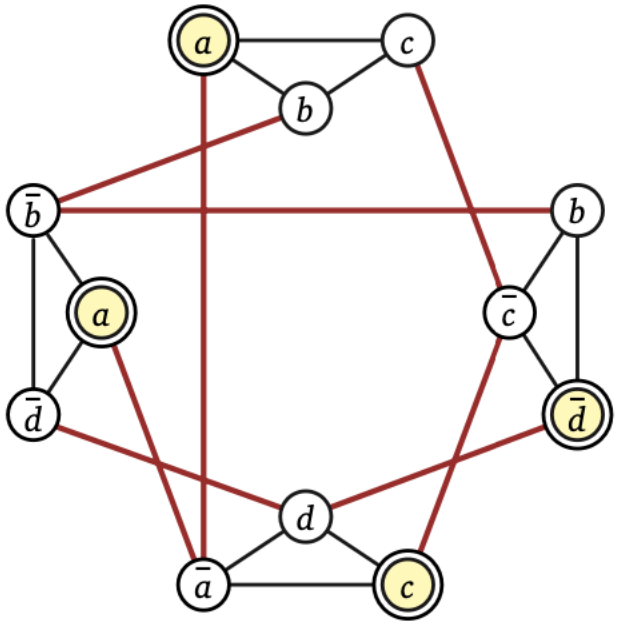


3SAT и максимальное независимое множество графа

- Пусть задан произвольный граф G.
- Необходимо найти максимальное независимое множество вершин в заданном графе.
- Теорема. G содержит независимое множество вершин размером ровно k, если и только если исходная формула Φ является выполнимой (*необходимость* и *достаточность*).

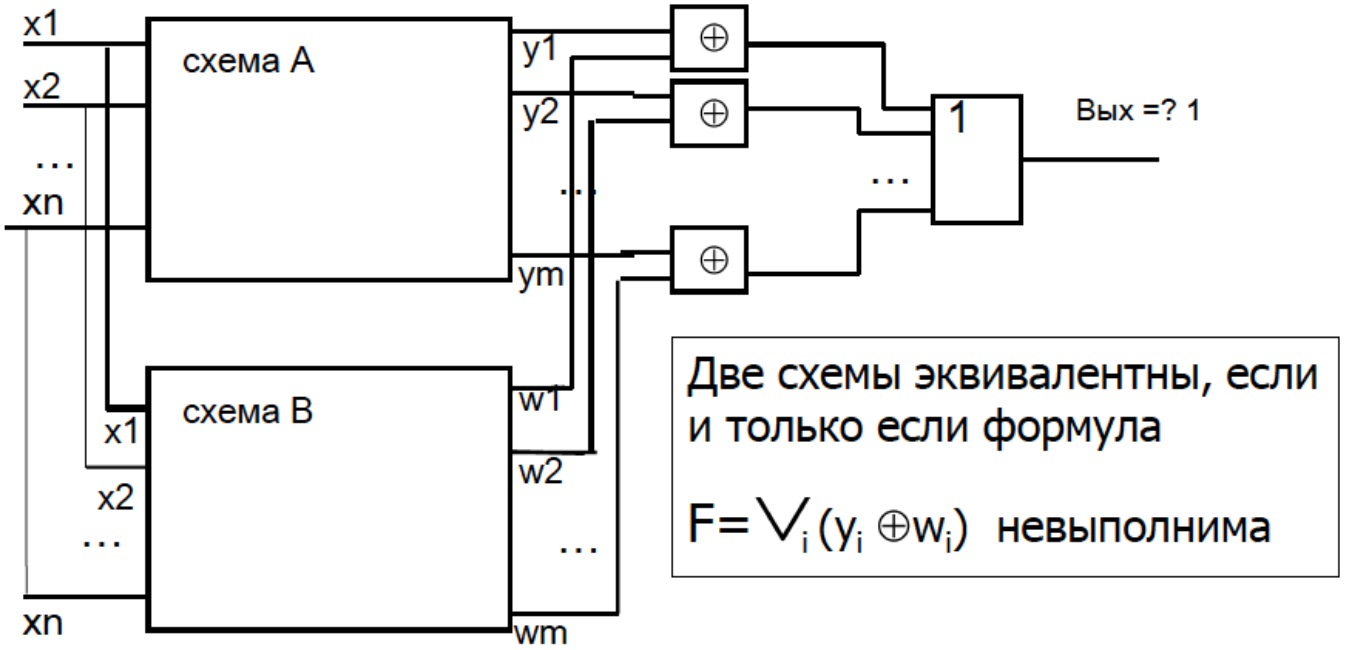


3SAT и максимальное независимое множество графа



$$(a \vee b \vee c) \wedge (b \vee \bar{c} \vee \bar{d}) \wedge (\bar{a} \vee c \vee d) \wedge (a \vee \bar{b} \vee \bar{d})$$

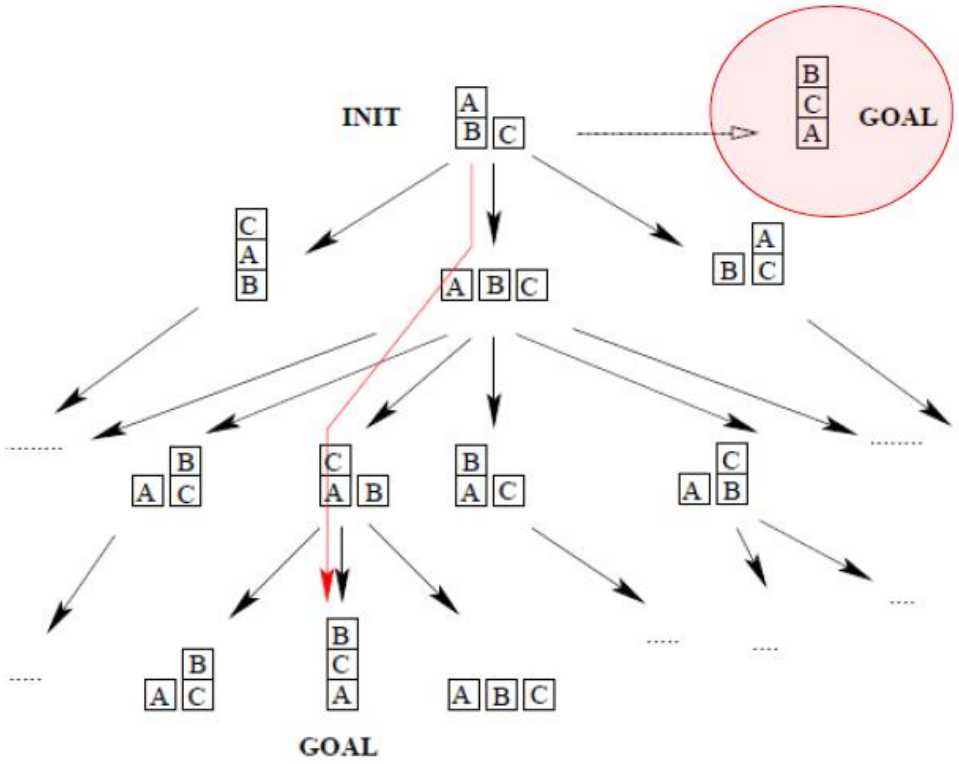
Проблема эквивалентности комбинационных схем сводится к SAT



- Строим $F=\bigvee_i (y_i \oplus w_i)$ для композиции двух схем. Если F равна 1 хотя бы на одном наборе входных переменных, то хотя бы на одном выходе значения, полученные при работе схем, не совпадают

Проблема эквивалентности двух схем сводится к проблеме SAT!

Проблема планирования в ИИ. Мир блоков



- Проблема планирования может быть сведена к проблеме SAT

SAT для CSP (Constraint Satisfaction Problem)

- Пример: проверить выполнимость f с заданным множеством clauses (клозов):
  $f=(x_1 \vee \neg x_2 \vee \neg x_3 \vee x_4) \wedge (x_4 \vee x_5) \wedge (x_1 \vee \neg x_3 \vee \neg x_4 \vee x_4) \wedge (x_3 \vee \neg x_4 \vee \neg x_5)$

  Проблема SAT имеет много хорошо разработанных методов решения. *Современные SAT-решатели работают с миллионами клозов и сотнями двоичных переменных*

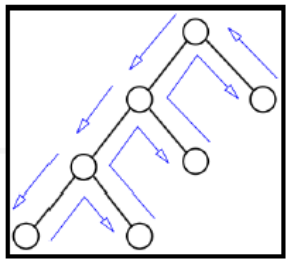
- Сложность проверки выполнимости, невыполнимости и т.д. существенно зависит от вида формулы. Сложность проверки выполнимости формулы, представленной в ДНФ, полиномиальна:
  Пример: проверить выполнимость формулы:
  $x_1 \neg x_2 \neg x_3 x_4 \vee x_4 \neg x_5 \vee x_1 x_3 \neg x_4 x_4 \vee x_3 \neg x_4 \neg x_5$ – выполнима, если хотя бы в одном конъюнкте нет пары контрарных литер

Более широкая проблема: CSP (Constraint Satisfaction Problem) – *выполнимость при ограничениях* (могут ли выполняться одновременно все ограничения, накладываемые на переменные)

Проблема SAT является центральной при решении проблемы CSP

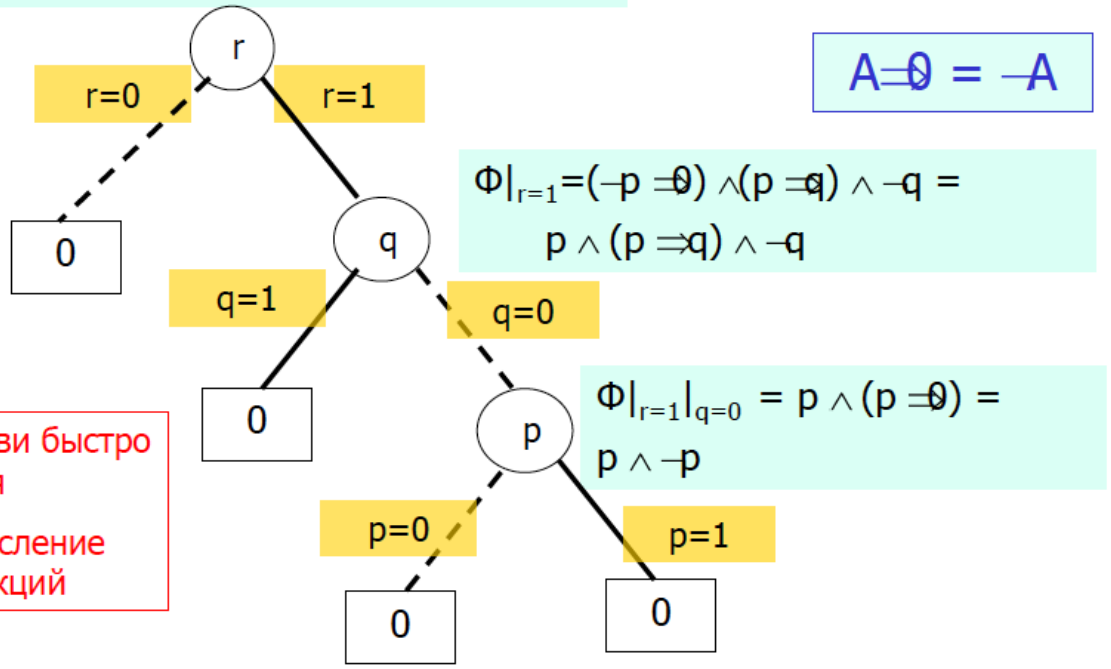
Метод Квайна – "splitting algorithm"

Строим решающее дерево, в узлах которого – частичные функции



Обход дерева

$\Phi = (\neg p \Rightarrow r) \wedge (p \Rightarrow q) \wedge r \wedge \neg q$



$A \Rightarrow 0 = \neg A$

$\Phi|_{r=1} = (\neg p \Rightarrow 0) \wedge (p \Rightarrow q) \wedge \neg q =$
$p \wedge (p \Rightarrow q) \wedge \neg q$

$\Phi|_{r=1}|_{q=0} = p \wedge (p \Rightarrow 0) =$
$p \wedge \neg p$

Основная идея – ветви быстро обрываются

Неудобство: вычисление частичных функций

Метод Девиса-Патнема проверки SAT

- Вход – CNF (конъюнктивная нормальная форма):
  $F=(x \vee y \vee \neg z) \wedge (\neg x \vee t) \wedge (\neg y \vee z \vee \neg t) \wedge t \wedge (\neg z \vee \neg w \vee \neg t) \wedge (\neg z \vee \neg t) \wedge (z \vee y)$

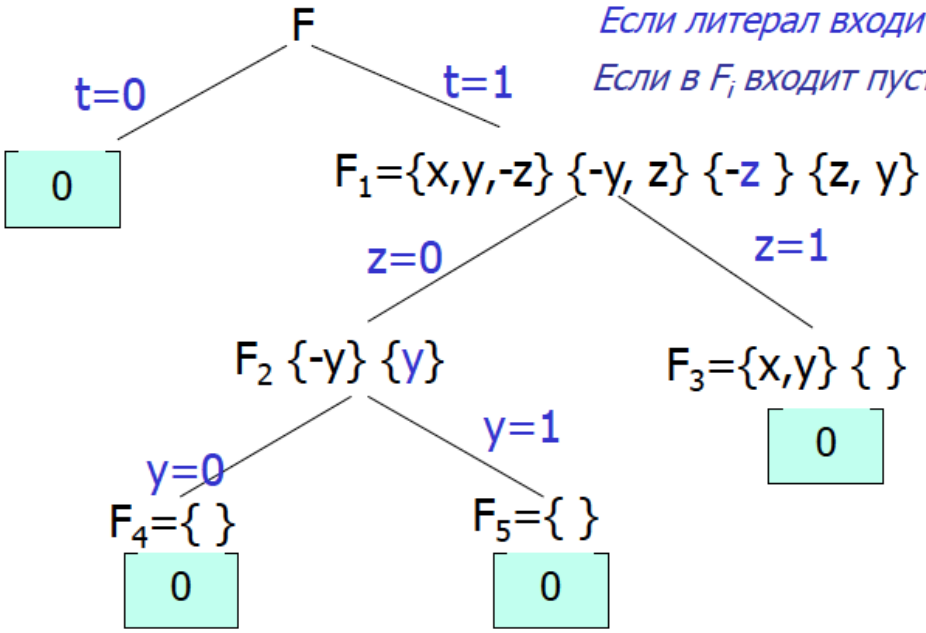
Основная идея – представить дизъюнкты множествами наборов литералов

$F = \{x,y,-z\}\ \{-x,t\}\ \{-y, z,-t\}\ \{t\}\ \{-z,-w, t\}\ \{-z,-t\}\ \{z, y\}$

Если литерал входит, как 1, то клоз выбрасывается

Если литерал входит, как 0, то литерал выбрасывается

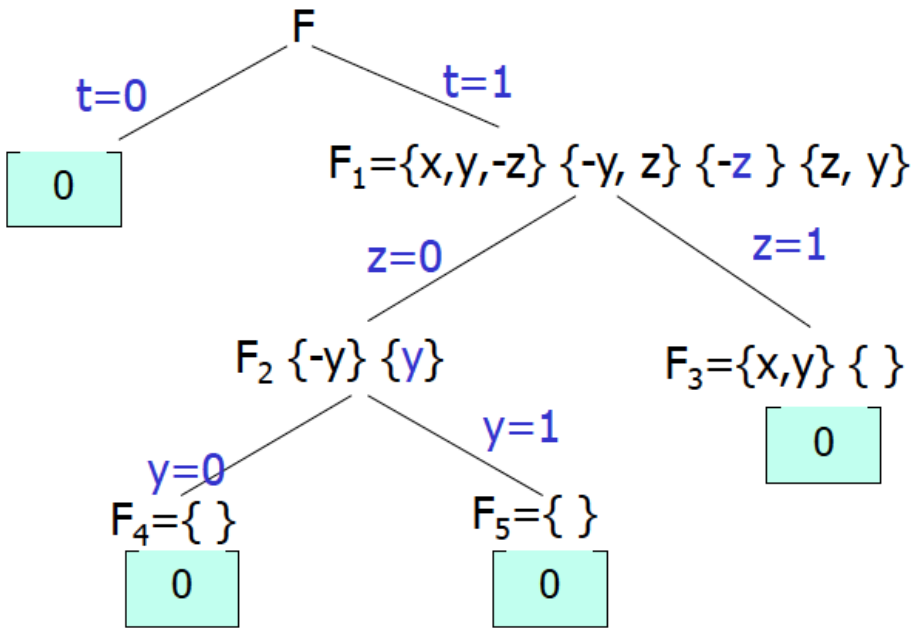
Если в F_i входит пустой клоз { }, то $F_i=0$



DPLL алгоритм (Davis, Putnam, Logemann, Loveland)

- Модификация DP алгоритма введением двух эвристик: "чистый литерал" и "единичный клоз"

F= {x,y,-z} {-x,t} {y, z,-t} {t} {-z,-w, t} {-z,-t} {z, y}



"Чистый литерал" – если переменная во все клозы входит только с одним знаком – с отрицанием, или без. "Чистым" литералам можно задать значение 1, и все содержащие их дизъюнкты станут истинными. Все эти дизъюнкты можно удалить.

Фактически, для чистого литерала L функция F заменяется на F∧L

"Единичный клоз" – клоз содержит только только L. Литералу L надо присвоить 1, и представить функцию F без этого клоза

Проблема SAT: Алгоритм DPLL

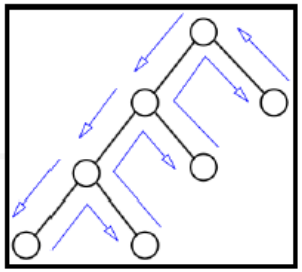
Алгоритм Дэвиса–Патнема–Логемана–Лавленда (DPLL) - это полный алгоритм поиска с возвратом для определения выполнимости булевых формул, т.е. для решения задачи CNF-SAT

DPLL - это усовершенствование более раннего алгоритма, разработанного Дэвисом и Патнемом. DPLL - высокоэффективный алгоритм, основа для большинства современных решателей для SAT и систем автоматического доказательства теорем

Две эвристики: "чистый литерал" и "единичный клоз"

Вход: Набор дизъюнктов формулы Ф
Выход: Значение истинности Ф

Алгоритм DPLL:
function DPLL(Ф) {
  if Ф - это набор выполняющихся дизъюнктов **then return** true;
  if Ф содержит пустой дизъюнкт **then return** false;
  для каждого чистого литерала *L* в Ф: Ф←*pure-literal-assign* (*L*, Ф);
  для каждого дизъюнкта из одного литерала *L* в Ф: Ф←*unit-propagate* (*L*, Ф);
  L←*choose-literal*(Ф);
return *DPLL*(Ф&L) **or** *DPLL*(Ф&not(L));
}



Проблема SAT – метод резолюции

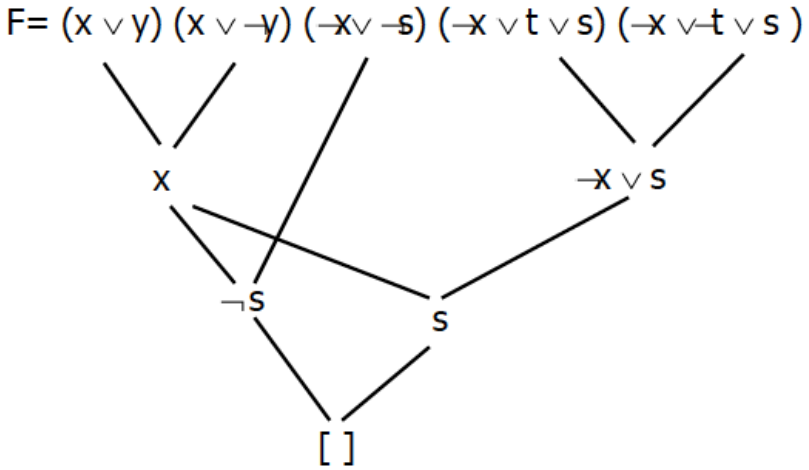
Определение. Резольвента двух дизъюнктов $(\lambda \vee D)$ и $(\neg\lambda \vee D1)$ - дизъюнкт $D \vee D1$

Пример: Резольвента дизъюнктов $(x \vee \neg y)$ и $(\neg x \vee t \vee s)$ - дизъюнкт $(\neg y \vee t \vee s)$

ТЕОРЕМА. Если R - резольвента дизъюнктов D и D1, то $D \wedge D1 = D \wedge D1 \wedge R$
(конъюнктивное присоединение резольвенты к конъюнкции порождающих ее дизъюнктов не изменяет исходную конъюнкцию)

Пример: Формулы $(x \vee \neg y) \wedge (\neg x \vee t \vee s)$ И $(x \vee \neg y) \wedge (\neg x \vee t \vee s) \wedge (\neg y \vee t \vee s)$ эквивалентны

Решение проблемы SAT методом резолюции



- Каждая построенная резольвента может быть конъюнктивно присоединена к исходной формуле в КНФ
- Поскольку получена пустая резольвента, имеющая значение 0, то вся формула F имеет значение 0

Проблема SAT: Хорновские дизъюнкты

- Дизъюнкт D называется Хорновским, если в него входит без отрицания не более одной переменной, например:

  $D = \neg x_1 \vee x_2 \vee \neg x_3$

- Формула называется Хорновской, если она представлена в КНФ и все ее дизъюнкты Хорновские

  Пример хорновской формулы:

  $(\neg x1 \vee \neg x2 \vee x3) \wedge (\neg x1 \vee x2 \vee \neg x4) \wedge (\neg x2 \vee \neg x4) \wedge x3$

- Хорновский дизъюнкт эквивалентно представим импликацией:

  $(x_1 \wedge \ldots \wedge x_n) \Rightarrow x$ или $(x_1 \wedge \ldots \wedge x_n) \Rightarrow 0$ (почему?)

Теорема. Сложность проблемы выполнимости Хорновской формулы полиномиальна (самостоятельно!)



2-SAT: полиномиальный алгоритм

- Проблема 2-SAT– это частный случай проблемы SAT, при котором функция F представлена множеством клозов, состоящих из не более, чем двух литералов
- Пример. $F= (x_1 \vee \neg x_2) \wedge (x_3 \vee \neg x_2) \wedge (x_1)$
- Теорема. Сложность решения 2-SAT проблемы линейна
- Доказательство.
- (самостоятельно)



Проблема SAT – одна из наиболее интересных проблем логики

- Ежегодно проводится International Conference on Theory and Applications of Satisfiability Testing. Конференция проводит "SAT Competition", которое оценивает новые пакеты решающие проблему SAT (SAT solvers)
- В Нидерландах - научный журнал JSAT (Journal on Satisfiability, Boolean Modeling and Computation) , посвященный этой проблеме и ее приложениям
- Ежегодно соревнуются десятки SAT решателей, соревнования состоят в решении тысяч "benchmark problems" (типовых проблем). Большинство подобных солверов можно скачать и использовать бесплатно:
  - CHAFF (Princeton Uni) http://www.princeton.edu/~chaff/
  - HyperSAT http://www.domagoj-babic.com/index.php/ResearchProjects/HyperSAT
  - MiniSAT http://minisat.se/
  - UBCAST http://www.satlib.org/ubcsat/
  - RSat (UCLA) http://reasoning.cs.ucla.edu/rsat/home.html
  - GRASP http://en.wikipedia.org/wiki/GRASP_(SAT_solver)
  - ... ...
- Смотри сайт SAT Live! (http://www.satlive.org) и SATLIB http://www.satlib.org
- Задача. Сформулируйте проблему расстановки 8 ферзей на доске 8x8 так, чтобы они не били друг друга



Применение двоичных функций: представление конечных множеств, конечных функций и отношений, а также операций над ними, с помощью характеристических БФ

Символьное представление



Символьное представление конечных отображений (функций)

- Любая конечная функция может быть представлена (и реализована) с помощью двоичных функций

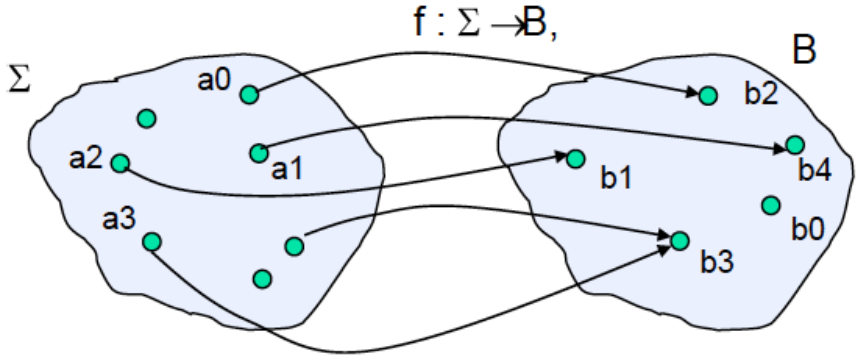


Таблица конечной функции

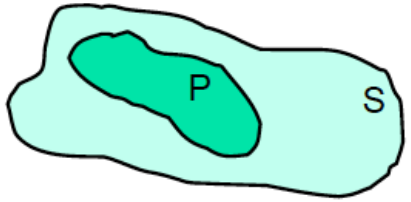
x	f(x)
a0	b2
a1	b4
a2	b1
a3	b3
a4	b3

После двоичного кодирования множества X аргументов (m двоичных аргументов) и множества У результатов (n двоичных аргументов) можно получить представление конечного отображения либо набором из n двоичных функций от m двоичных переменных, либо одной двоичной функцией от m+n двоичных переменных

Конечные структуры данных: характеристическая функция подмножеств конечного множества

$x_1x_2x_3$	f
000	1
001	1
010	1
011	0
100	0
101	0
110	1
111	0

Пусть S – конечное множество из m элементов, $k=\lceil \log_2(m) \rceil$. Закодируем все элементы S наборами двоичных переменных $x_1, x_2, \ldots, x_k$.



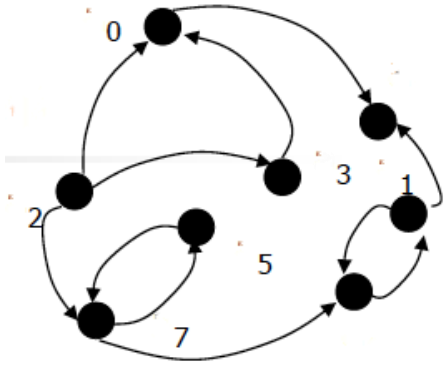
Пример: S - из 8 элементов: $\{a_0, a_1, \ldots, a_7\}$.
Кодируем (k=3): $a_0 \Leftrightarrow 000$, $a_1 \Leftrightarrow 001$, … , $a_7 \Leftrightarrow 111$

Пусть $P \subseteq S$, $P= \{a_0, a_1, a_2, a_6\}$. Соответствующее множество двоичных кодов { 000, 001, 010, 110}.

Булева функция f может задать множество элементов - подмножество конечного множества, если она равна 1 на наборах, кодирующих элементы, которые в это множество входят.
Эта функция называется характеристической функцией множества.
Обозначим χ_P характеристическую функцию множества P.

Любое подмножество конечного множества может быть задано с помощью характеристической БФ. Такое задание называется символьным представлением множества

Символьное представление отношений



Пусть $S=\{a_0, \dots, a_{n-1}\}$ – множество

Введем кодирование: $a_0 \Leftrightarrow 000$, $a_1 \Leftrightarrow 001$, … ,$a_7 \Leftrightarrow 111$

Бинарное отношение на S – подмножество пар из S, $R= \{ (a_2,a_3), (a_4,a_7), (a_5, a_7), \dots\}$

Первый элемент отношения R – это текущая вершина (pre-state), разряды ее кода обозначим $v =(x_1, x_2, x_3, \dots)$.

Второй элемент отношения – (post-state) и ее код – $v' = (x_1', x_2', x_3', \dots)$

χ_R - характеристическая функция R равна 1 на кодировках $\{(a_0,a_4), (a_2,a_0), (a_2,a_7)\dots\}$, т.е. на наборах (v, v') 6 булевых переменных { 000100, 010000, 010111, ... }

$$\chi_R = \neg x_1 \neg x_2 \neg x_3 \neg x_1' x_2' x_3' \vee$$
$$\neg x_1 x_2 \neg x_3 \neg x_1' \neg x_2' \neg x_3' \vee$$
$$\neg x_1 x_2 \neg x_3 x_1' x_2' x_3' \vee \dots$$

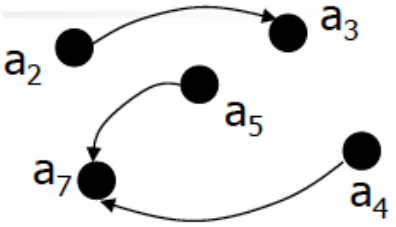
Характеристическая функция отношения обычно определяется как логическая формула над штрихованныими и нештрихованными двоичными переменными

Ее можно представить формулами, ТИ, а также в любой другой форме

Операции над отношениями

Пусть $S=\{a_0, \dots, a_{n-1}\}$ – множество

Кодирование: $a_0 \Leftrightarrow 000$, $a_1 \Leftrightarrow 001$, … ,$a_7 \Leftrightarrow 111$



Бинарное отношение на S – подмножество пар из S, $R= \{ (a_2,a_3), (a_4,a_7), (a_5, a_7) \}$

Характеристическая функция χ_R отношения $R= \{ (a_2,a_3),(a_4,a_7), (a_5, a_7) \}$ в СДНФ:

$\chi_R = (\neg x_1 x_2 \neg x_3 \neg x_1' x_2' x_3') \vee (x_1 \neg x_2 \neg x_3 x_1' x_2' x_3') \vee (x_1 \neg x_2 x_3 x_1' x_2' x_3')$

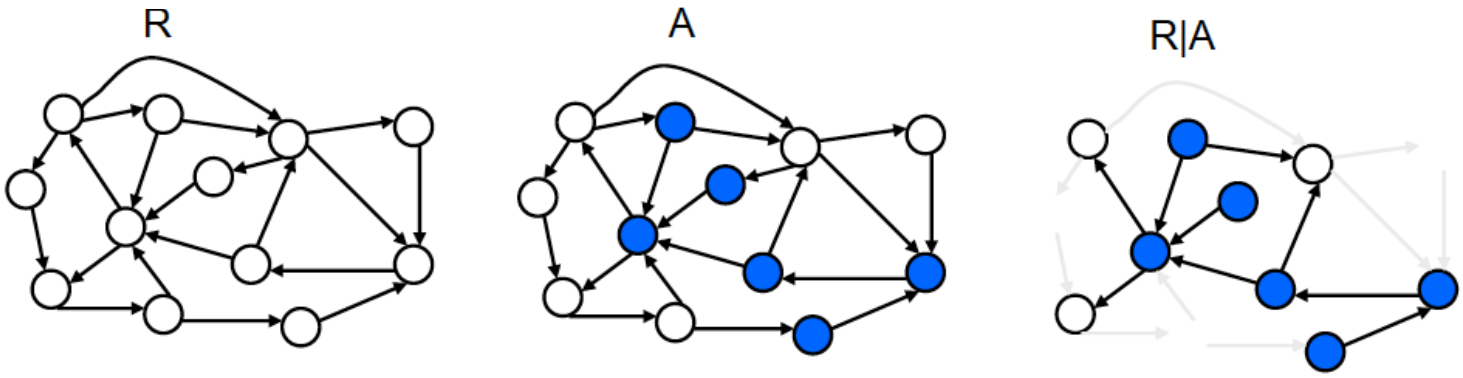
$(\exists v).\chi_R$ – получение множества всех тех элементов, которые закодированы штрихованными переменными, т.е. вторых элементов отношения R

$(\exists v).\chi_R = \neg x_1' x_2' x_3' \vee x_1' x_2' x_3' \vee x_1' x_2' x_3'$ – кодировка V' двух переменных: $\{a_2, a_7\}$

$(\exists v').\chi_R$ – получение множества всех тех элементов, которые закодированы НЕштрихованными переменными, т.е. первых элементов отношения R

$(\exists v').\chi_R = \neg x_1 x_2 \neg x_3 \vee x_1 \neg x_2 \neg x_3 \vee x_1 \neg x_2 x_3$ – кодировка V' переменных: $\{a_2, a_4, a_5\}$

Ограничение отношения на подмножестве



Пусть R – бинарное отношение на S, и A – подмножество S

Построим ограничение отношения R на множестве A (обозначается R|A)

Это такие пары отношения R, первые элементы которых входят в A

Как его построить эффективно при огромных размерах множеств???
(до 10^{100} и больше!!!) - используем BDD для χ_R - и χ_A – характеристических булевых функций отношения R и множества A

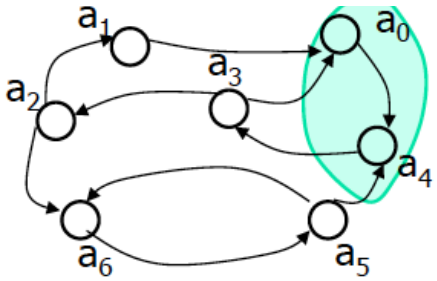
Характеристическая БФ отношения R|A строится как $\chi_A \wedge \chi_R$ – операция над двумя булевыми характеристическими функциями

Ограничение отношения на множестве

$A = \{ a_0, a_4 \}$

$x_1\, x_2\, x_3$

a_0 – 0 0 0
a_1 – 0 0 1
… …
a_6 – 1 1 0



$R = \{(a_0a_4), (a_1a_0), (a_2a_1), (a_2a_6), (a_3a_2), (a_3a_0), (a_4a_3), (a_5a_4), (a_5a_6), (a_6a_2)\}$

$R|_1A = \{ (a_0a_4), (a_4a_3) \}$

$R|_2A = \{ (a_1a_0), (a_3a_0), (a_0a_4), (a_5a_4) \}$

(V'/V) – переименование переменных

$$\chi_R = \neg x_1 \neg x_2 \neg x_3\, x'_1 \neg x'_2 \neg x'_3 \vee \neg x_1 \neg x_2 x_3 \neg x'_1 \neg x'_2 \neg x'_3 \vee \neg x_1 x_2 \neg x_3 \neg x'_1 \neg x'_2 x'_3 \vee \neg x_1\, x_2 \neg x_3\, x'_1 x'_2 \neg x'_3 \vee \ldots \vee x_1\, x_2 \neg x_3 \neg x'_1 x'_2 \neg x'_3$$

$$\chi_A = \neg x_1 \neg x_2 \neg x_3 \vee \neg x_1\, x_2\, x_3 \vee x_1 \neg x_2\, x_3$$

Ограничение1 R на A = $\chi_R \wedge \chi_A$

$$\chi_{R|1A} = \chi_R \wedge \chi_A = \neg x_1 \neg x_2 \neg x_3\, x'_1 \neg x'_2 \neg x'_3 \vee x_1 \neg x_2 \neg x_3 \neg x'_1\, x'_2\, x'_3$$

$$(\exists v).\chi_{R|1A} = x'_1 \neg x'_2 \neg x'_3 \vee \neg x'_1\, x'_2\, x'_3$$

все вторые элементы $R|_1A$

Ограничение2 R на A = $\chi_R \wedge \chi_A(v'/v)$

$$\chi_{R|2A} = \chi_R \wedge \chi_A(V'/V) = \neg x_1 \neg x_2\, x_3 \neg x'_1 \neg x'_2 \neg x'_3 \vee \neg x_1\, x_2\, x_3 \neg x'_1 \neg x'_2 \neg x'_3 \vee \neg x_1 \neg x_2 \neg x_3\, x'_1 \neg x'_2 \neg x'_3 \vee x_1 \neg x_2\, x_3\, x'_1 \neg x'_2 \neg x'_3$$

$$(\exists v').\chi_{R|2A}(v'/v) = \neg x_1 \neg x_2\, x_3 \vee \neg x_1\, x_2\, x_3 \vee \neg x_1 \neg x_2 \neg x_3 \vee x_1 \neg x_2\, x_3$$

все первые элементы $R|_2A$

Построение тестов для задач классификации

- Задача классификации множества объектов на два непересекающихся класса с бинарными признаками по заданным обучающим выборкам
- Признаковые описания объектов задаются матрицей «объект-признак»

объекты обучения	p1	p2	p3	p4	p5	
1	1	1	0	1	0	Объекты класса 1
2	0	1	1	0	1	
3	1	0	1	0	1	
4	1	0	0	1	1	Объекты класса 2
5	0	0	1	0	0	
6	0	1	0	1	1	

Тест – набор признаков, который для всех объектов двух разных классов различается. Примеры:
T1={p1,p2,p3,p4,p5} - тест,
T2={p4,p5} - тест,
T3={p2,p3,p4} – НЕ тест (об 1 и 6).
Тупиковый тест – такой тест, что никакое его собственное подмножество не является тестом.
T2 – тупиковый тест, T1 и T3 - нет

Интерпретация матрицы: В обучающем множестве объект 1 относится к классу 1, он характеризуется наличием признаков p1, p2 и p4 и отсутствием признаков p3 и p5.
Пусть описания объектов обучения из первого и второго классов не пересекаются (т.е. начальная информация непротиворечива: нет объектов, принадлежащих разным классам, но характеризующихся одинаковыми наборами признаков)
Задача: найти все тупмковые тесты, т.е. наборы признаков, которые определят, относится ли произвольно заданный объект к классу 1 или к классу 2.

Построение теста для задачи классификации: сведение к сокращенной ДНФ

признаки

объекты обучения	p1	p2	p3	p4	p5	
1	1	1	0	1	0	1 класс
2	0	1	1	0	1	
3	1	0	1	0	1	
4	1	0	0	1	1	2 класс
5	0	0	1	0	0	
6	0	1	0	1	1	

Строим матрицу сравнений:

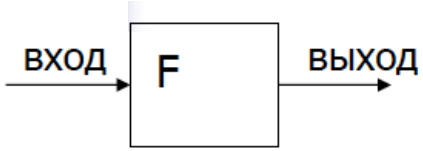
	p1	p2	p3	p4	p5
1⊕4	0	1	0	0	1
2⊕4	1	1	1	1	0
3⊕4	0	0	1	1	0
1⊕5	1	1	1	1	0
2⊕5	0	1	0	0	1
3⊕5	1	0	0	0	1
1⊕6	1	0	0	0	1
2⊕6	0	0	1	1	0
3⊕6	1	1	1	1	0

p1	p2	p3	p4	p5
0	1	0	0	1
1	1	1	1	0
0	0	1	1	0
1	0	0	0	1

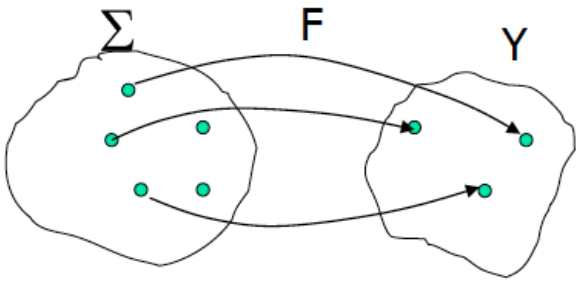
Неповторяющиеся строки матрицы сравнений

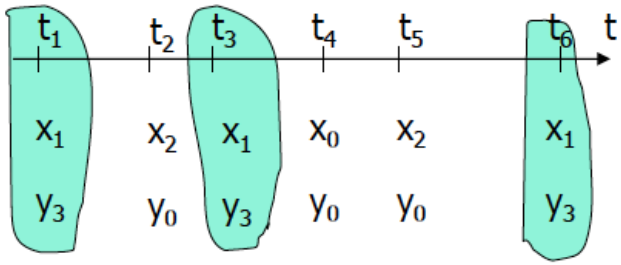
- Матрица сравнений: 1й и 4й (а также и 2й и 5й) элементы различаются по p2 и p5 (потому что в этих столбцах 1), 2й и 4й – по признакам p1, p2, p3 и p4, и т.д.
- По матрице сравнений строим тест, различающий элементы двух классов. В тест должна войти проверка по p2 или p5 (будут различены объекты 1 и 4) И проверка по p1 или p2 или p3 или p4 (при этом будут различены объекты 1 и 5) и т.п.
- Тест: $(p2 \vee p5)\ (p1 \vee p2 \vee p3 \vee p4)\ (p3 \vee p4)\ (p1 \vee p5) =$
  $p1p2p3 \vee p1p2p4 \vee p3p5 \vee p4p5$

Конечные функциональные преобразователи информации



Σ – конечное множество входных событий





Конечную функцию можно задать таблицей

Σ	f(x)
x_0	y_0
x_1	y_3
x_2	y_0
x_3	y_3
x_4	y_2

pqr	f_1f_2
000	00
001	11
010	01
011	11
100	10

Используем двоичное кодирование

x_0-000 y_0 - 00
x_1-001 y_1 – 01
x_2-010 y_2 – 10
x_3-011 y_3 - 11
x_4-100

Любое конечное отображение можно задать двоичными функциями, свести к двоичным функциям некоторого базиса и реализовать схемно

Два разных способа представления функции на конечном множестве с помощью двоичных функций

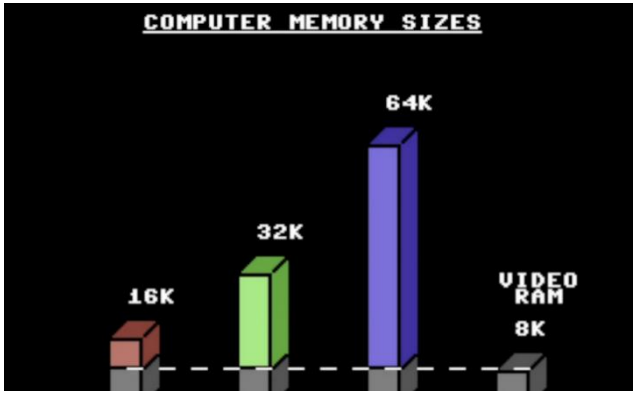


Иные способы использования двоичных функций

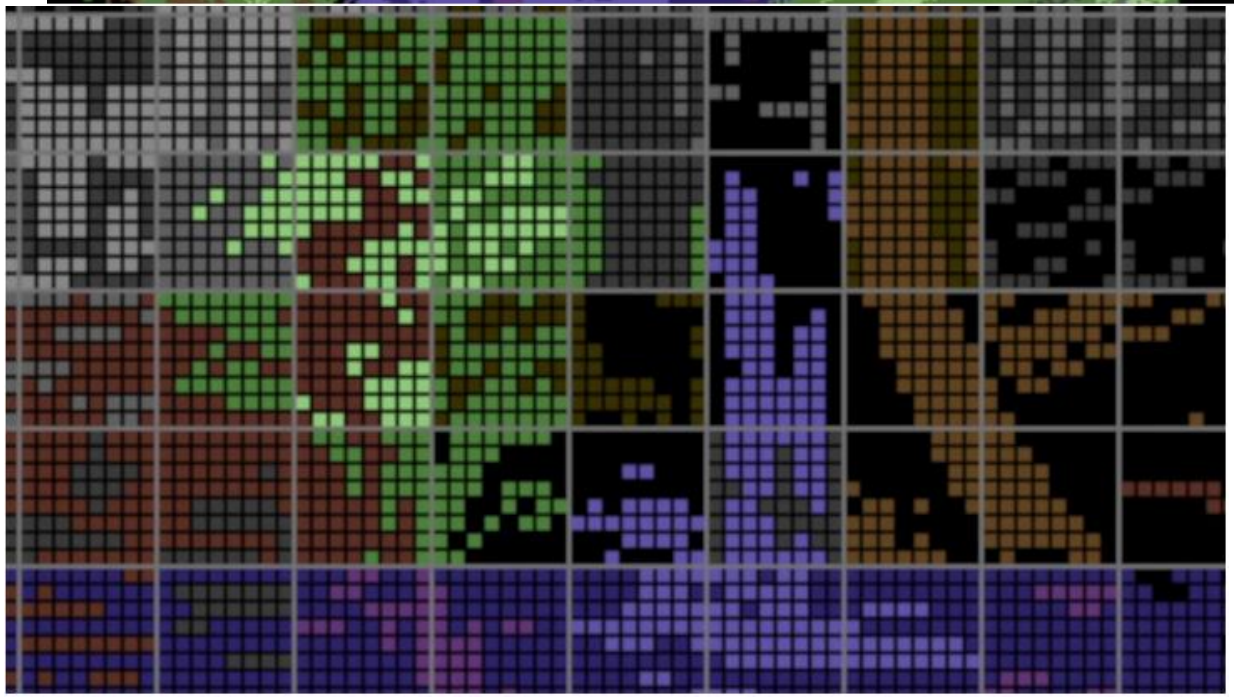


Двоичные функции в компьютерной графике (оптимизация на «заре времен»)

- Сперва разработчики наносили на экран монохромное изображение, в котором белые пиксели были условным передним планом, а черные – фоном. Затем весь экран делился на ячейки 8 на 8 пикселей. Для ячейки можно было задать только два цвета, зато каждая из них занимала только 1 байт памяти. Цветное изображение, созданное методом ячеек цвета, занимало всего 9 килобайт памяти – немногим больше, чем черно-белое.
- Все детали идеально распределялись по ячейкам цвета – ни в одной нет больше двух цветов!
- При этом у разработчиков была возможность пожертвовать разрешением, объединив пиксели в пары по горизонтали. Таким образом изображение становилось менее четким, но зато в каждой ячейке памяти можно было использовать уже 4 цвета, на это требовались те же 9 килобайт памяти.





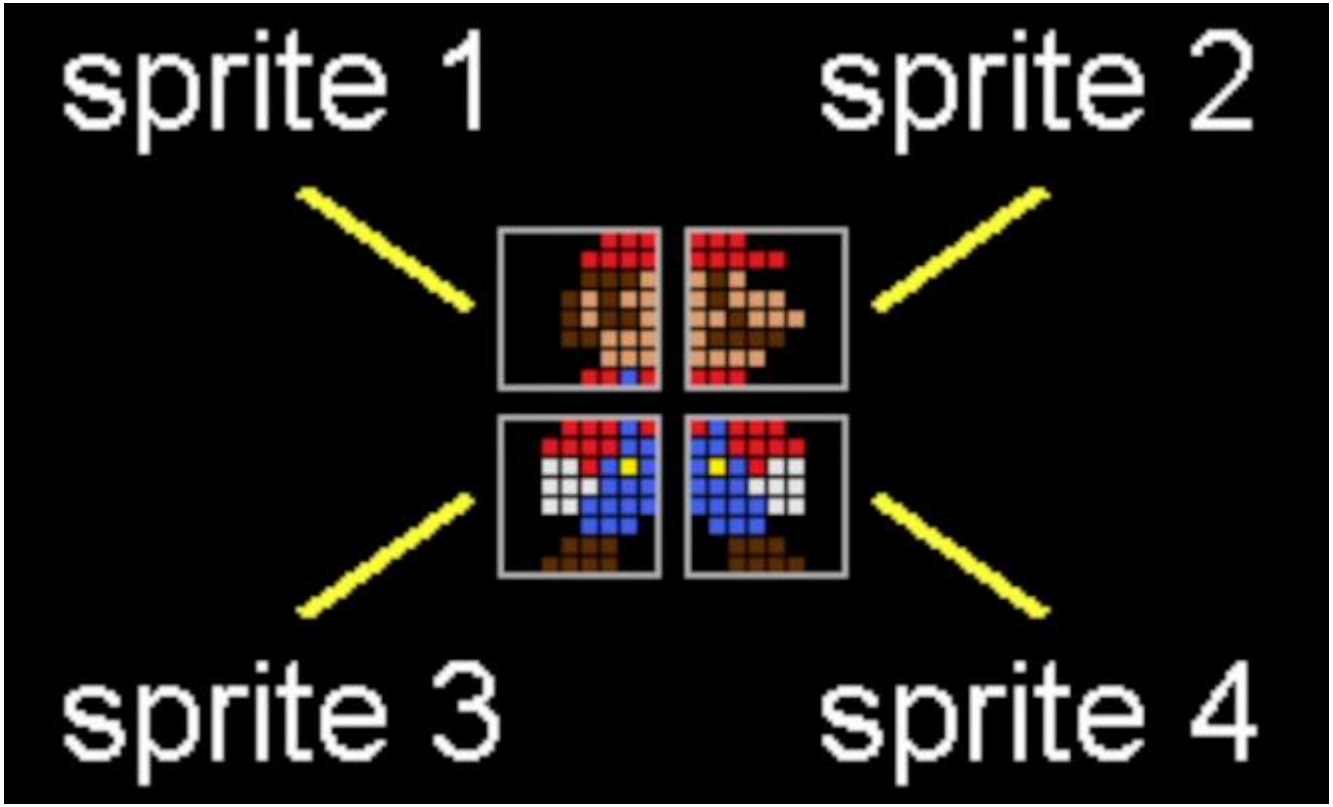



STE'86



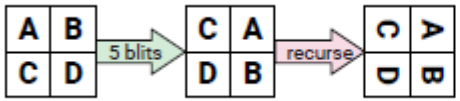
Аппаратное создание спрайтов

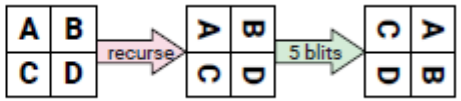
- Необходимо нарисовать изображение, конвертировать его в двоичный код и с помощью команды нарисовать на экране. Такое изображение существует вне зависимости от ячеек цвета. На экране - до 64 спрайтов, но они были очень маленькими.

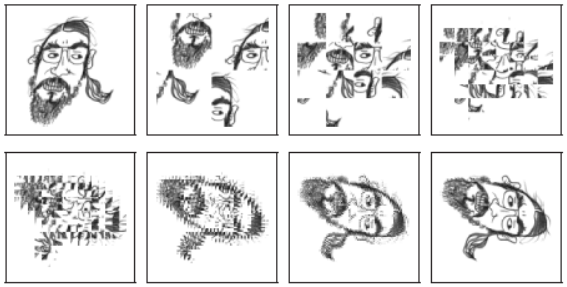


Два алгоритма для поворота карты пикселей

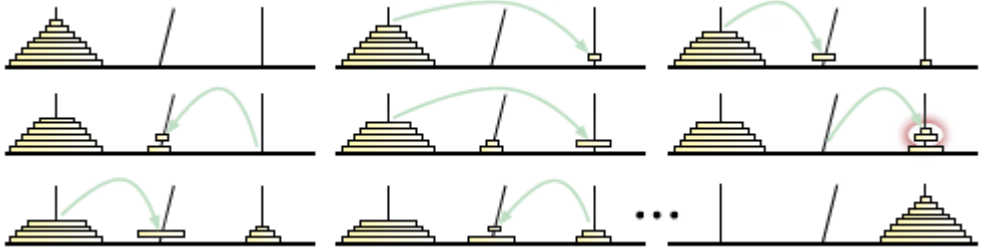
- Бо'льшая часть графических аппаратных средств включает поддержку низкоуровневой операции blit (block transfer), которая быстро копирует прямоугольный фрагмент карты пикселов (двумерный массив значений пикселов) из одной локации (памяти) в другую. Например, двумерная версия функция стандартной библиотеки языка C memcpy().
- Необходимо повернуть карту пикселов размером n*n на 90◦ по часовой стрелке:
  - разделяем карту пикселей на четыре блока размером n/2*n/2,
  - перемещаем каждый блок в требуемую позицию с использованием последовательности из 5 бит,
  - каждый блок рекурсивно поворачиваем.





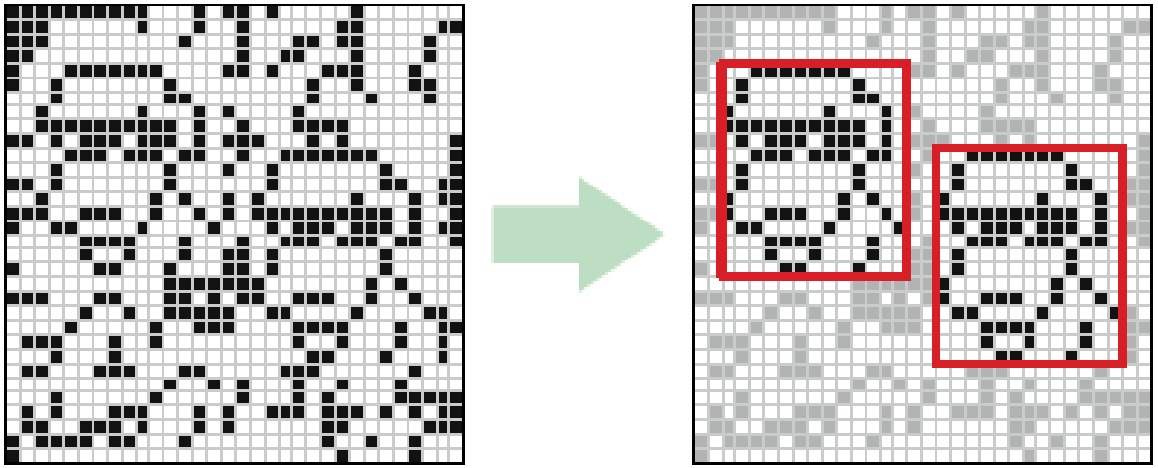


Почему 5 бит?



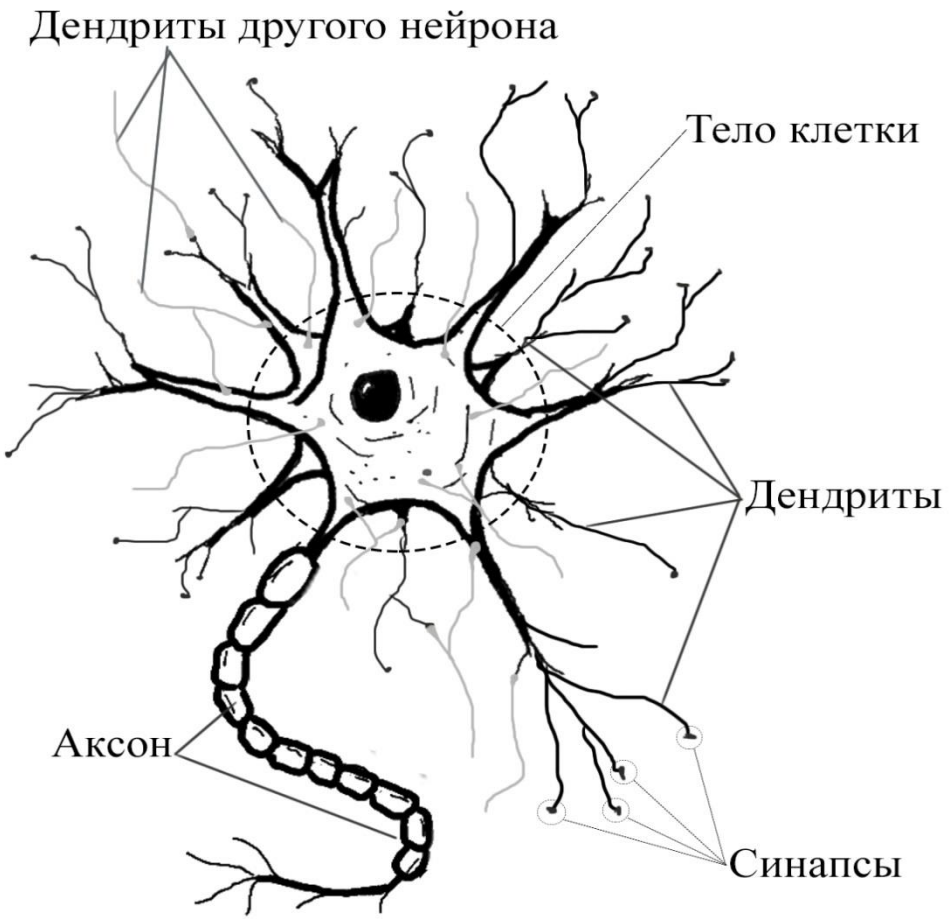
Алгоритм поиска прямоугольного образца максимальной площади

- Встречается не менее одного раза на заданной битовой карте.
- Задан двумерный массив битов M[1..n,1..n].
- Нужно найти значение площади максимального повторяющиеся прямоугольного образца в массиве M.

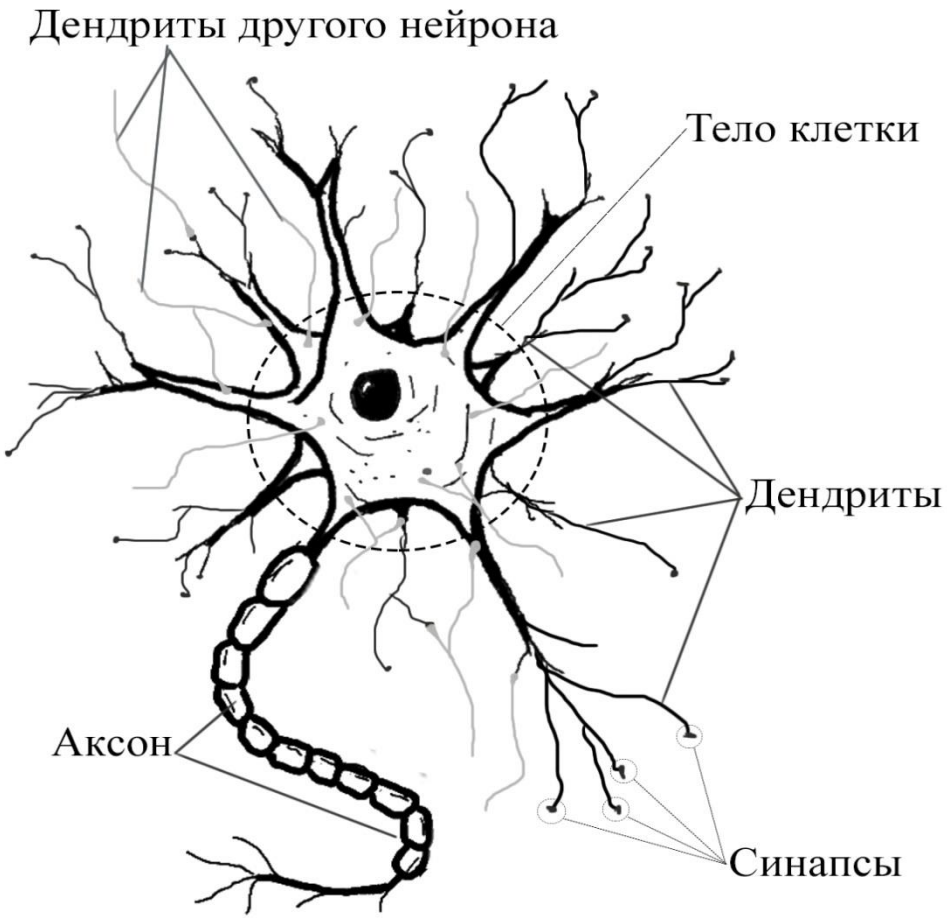


Ответ: **195** (площадь 15x13 данного песика)

Нейроны

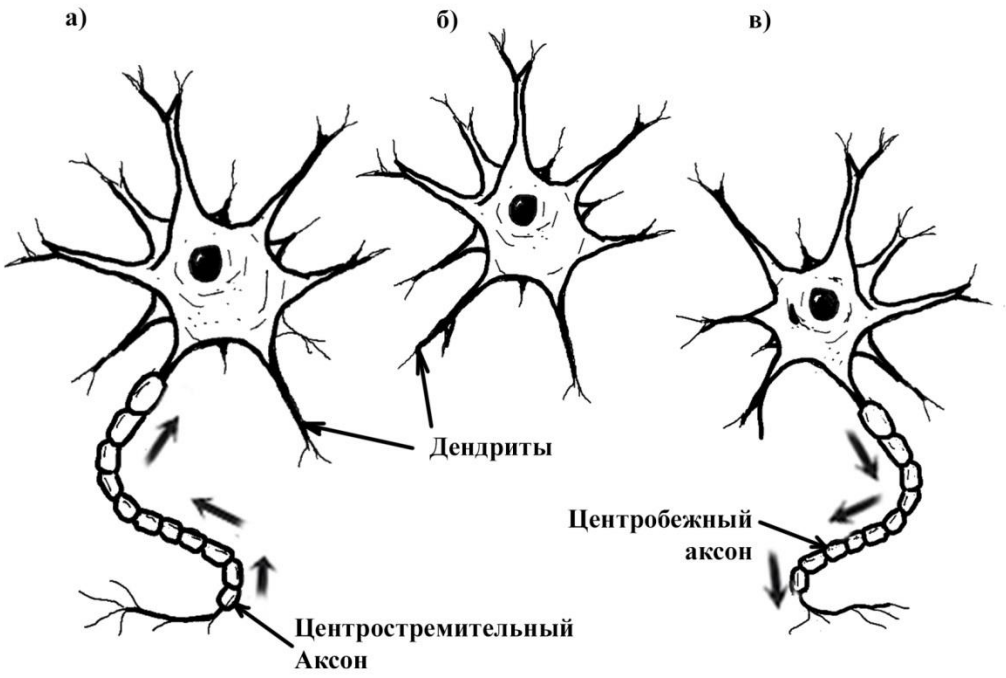

Дендриты другого нейрона
Тело клетки
Дендриты
Аксон
Синапсы

Нейрон



- Нейрон является основным функциональным элементом нервной системы, он состоит из тела нервной клетки и её отростков.
- Существуют два вида отростков: аксоны и дендриты.
- Аксон – длинный покрытый миелиновой оболочкой отросток, предназначенный для передачи нервного импульса на далекие расстояния.
- Дендрит – короткий, ветвящийся отросток, благодаря которым происходит взаимосвязь с множеством соседних клеток.

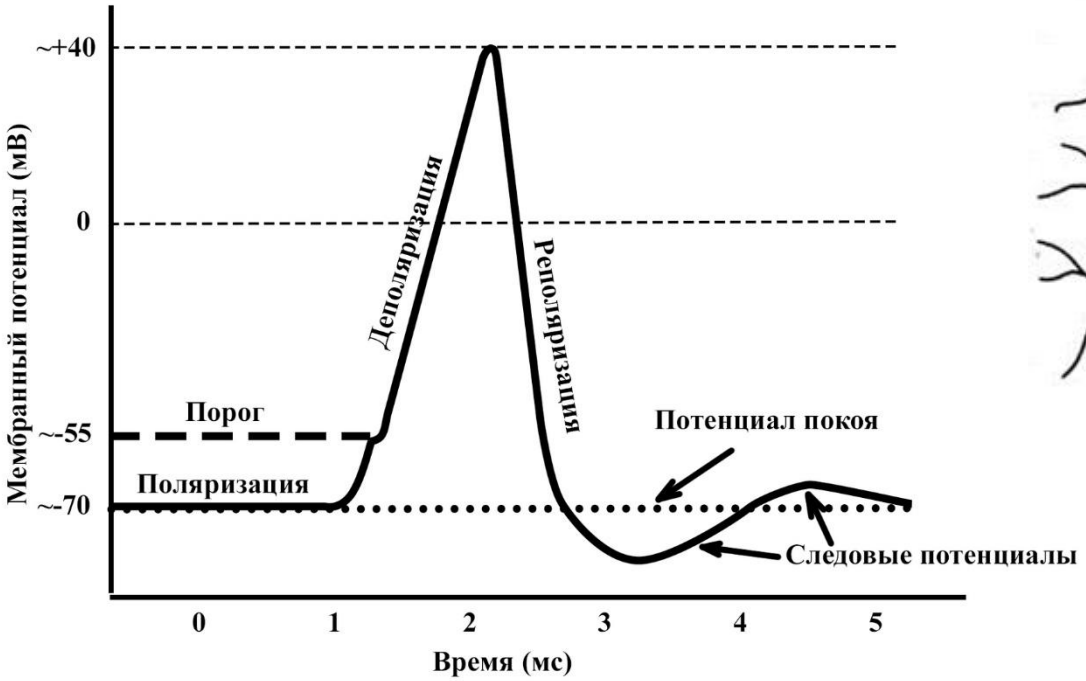
Три типа нейронов

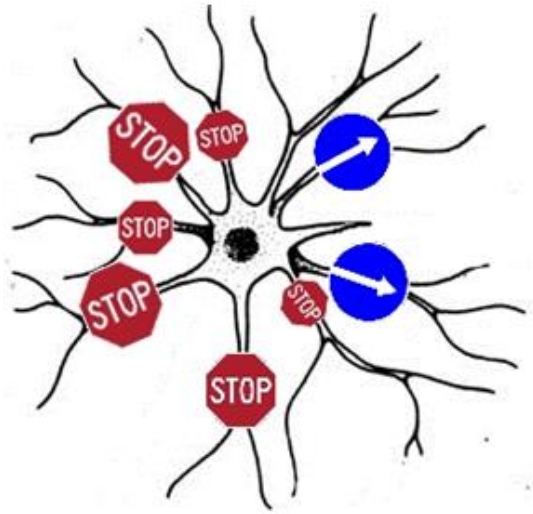


Обычно в статьях по искусственным нейронным сетям оговаривается наличие только моторных нейронов (с центробежным аксоном), которые связаны в слои иерархической структуры. Подобное описание применимо к биологической нервной системе, но является своего рода частным случаем, речь идет о структурах, базовых условных рефлексов. Чем выше в эволюционном значении нервная система, тем меньше в ней превалируют структуры типа «слои» или строгая иерархия.

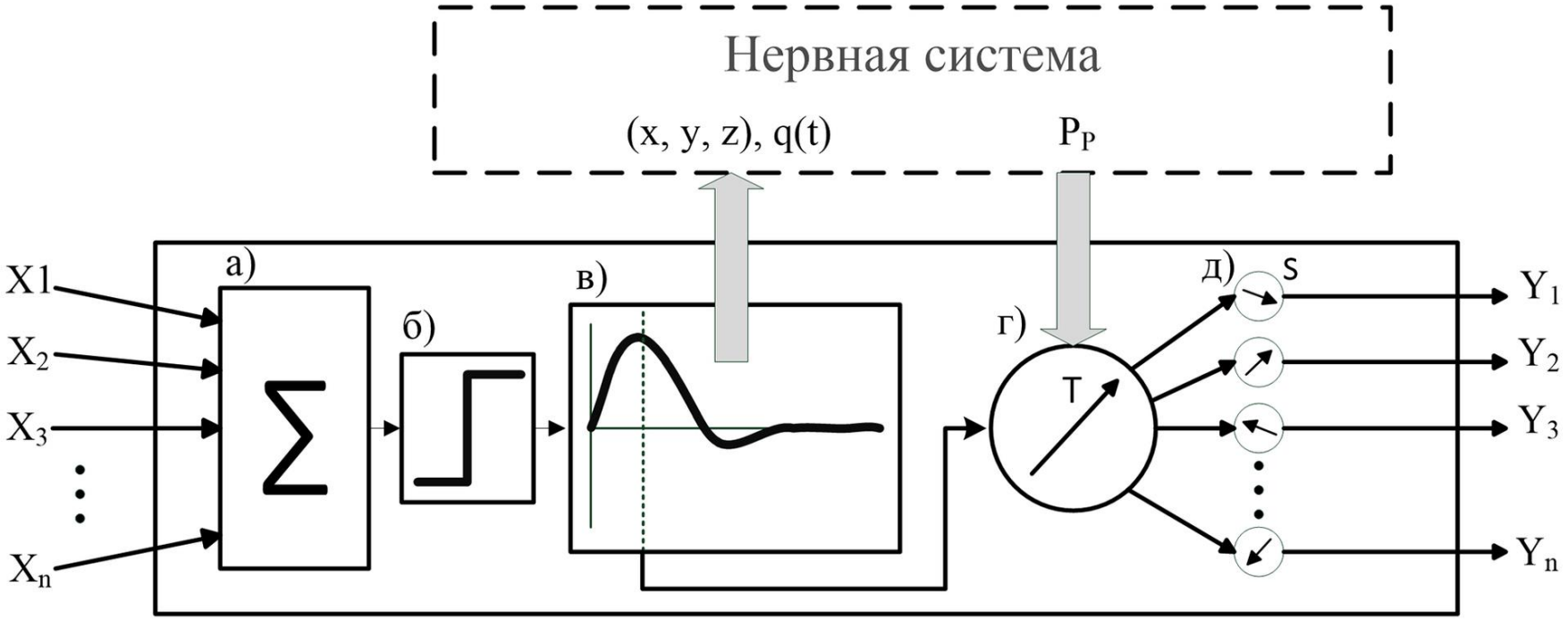
Состояние возбуждения нейрона

- В состоянии покоя мембрана нейрона поляризована. Это означает, что по обе стороны мембраны располагаются частицы, несущие противоположные заряды. В состоянии покоя наружная поверхность мембраны заряжена положительно, внутренняя – отрицательно.
- Разница между зарядами поверхности мембраны и внутри тела клетки составляет мембранный потенциал. Медиатор вызывает нарушения поляризации – деполяризацию. Положительные ионы снаружи мембраны устремляются через открытые каналы в тело клетки, меняя соотношение зарядов между поверхностью мембраны и телом клетки.





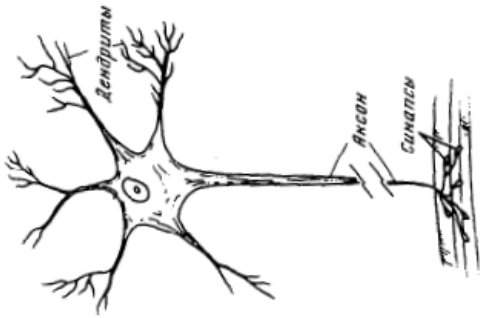
Математическая модель



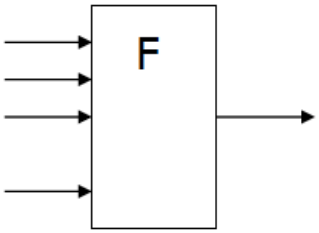
Математическая модель

- Входные сигналы (x_1, x_2,…x_n) нейрона представляют собой вещественные числа, которые характеризуют силу синапсов нейронов, оказывающих воздействие на нейрон.
- Положительное значение входа означает побудительное воздействие, оказываемое на нейрон, а отрицательное значение – тормозящее воздействие.
- Для биологического нейрона не имеет значение, откуда поступил возбуждающий его сигнал, результат его активности будет идентичен. Нейрон будет активизирован, когда сумма воздействий на него будет превышать определённое пороговое значение. Поэтому, все сигналы проходят через сумматор (а), а поскольку нейроны и нервная система работают в реальном времени, следовательно, воздействие входов должно оцениваться в короткий промежуток времени, то есть воздействие синапса имеет временный характер.
- Результат сумматора проходит пороговую функцию (б), если сумма превосходит пороговое значение, то это приводит к активности нейрона.
  При активации нейрон сигнализирует о своей активности системе, передовая информацию о своём положении в пространстве нервной системы и заряде, изменяемом во времени (в).
- Через определённое время, после активации нейрон передает возбуждение по всем имеющимся синапсам, предварительно производя пересчет их силы. Весь период активации нейрон перестает реагировать на внешние раздражители, то есть все воздействия синапсов других нейронов игнорируются. В период активации входит так же период восстановления нейрона.
- Происходит корректировка вектора T (г) с учётом значения точки паттерна Pp и уровнем нейропластичности. Далее происходит переоценка значений всех сил синапсов в нейроне(д).
  Обратите внимание, что блоки (г) и (д) выполняются параллельно с блоком (в).

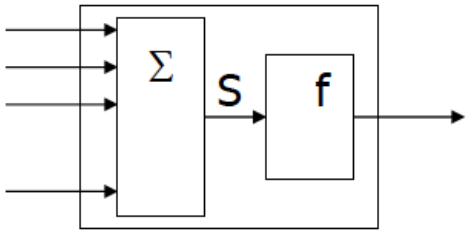
Формальные нейроны и двоичные функции



Формальный нейрон – двоичная функция



Обычно формальный нейрон Σ + пороговая функция

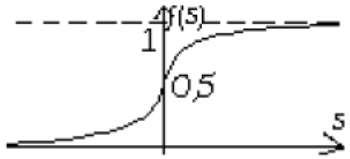


Σ- суммирование входов с весами $S=\Sigma_i (w_i x_i)$

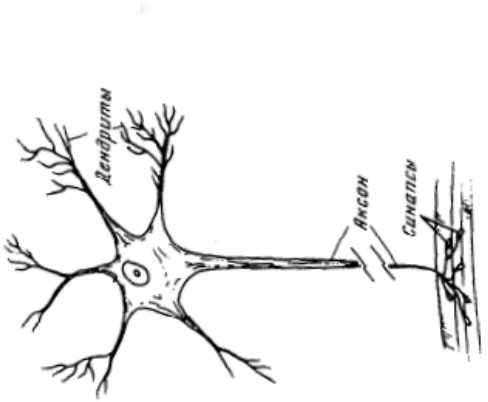
f - пороговая функция. Нейрон возбуждается, когда S (сумма возбуждений на его входах с некоторыми весами) превысит порог. Веса могут быть и отрицательными

- Существует много моделей нейронов. В простейшем случае f=1, если S ≥ T (порог), иначе f=0. Существуют более сложные модели:

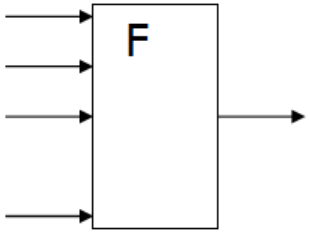
$$f(S) = \frac{1}{1 + e^{-\alpha S}}$$



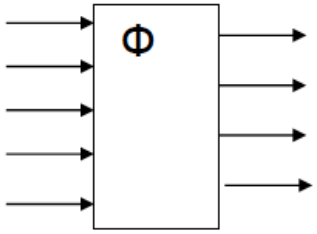
Формальные нейроны могут реализовывать любые двоичные функции



Формальный нейрон – модель двоичной функции
$F: \{0,1\}^n \to \{0,1\}$

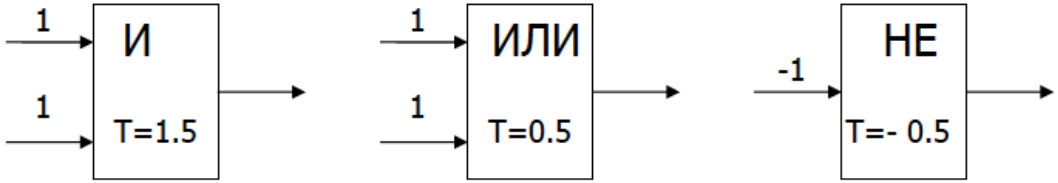


Нейронная сеть – модель отображения
$\Phi: \{0,1\}^n \to (0,1)^m$



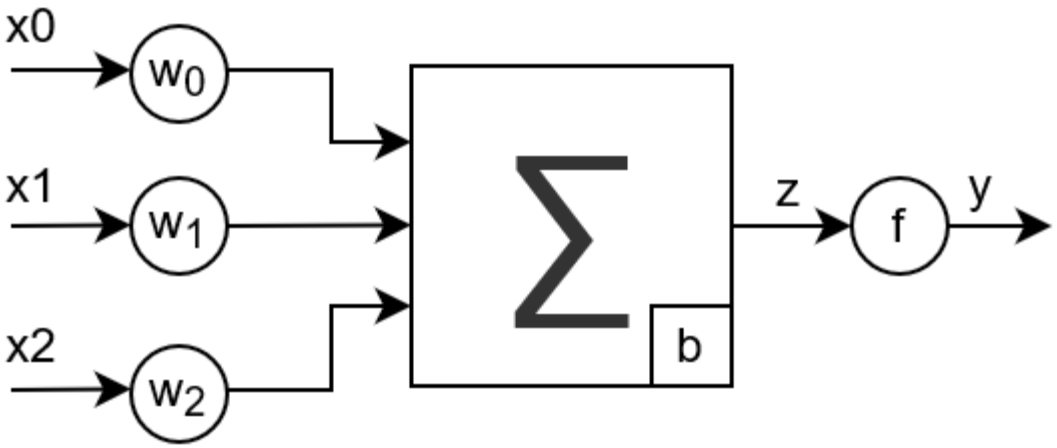
Можно ли из (формальных) нейронов построить полную систему функций?

T – treshold, порог



Комбинацией нейронов могут быть реализованы любые булевы функции

Математическая модель (упрощенная)



w_i - синаптические веса

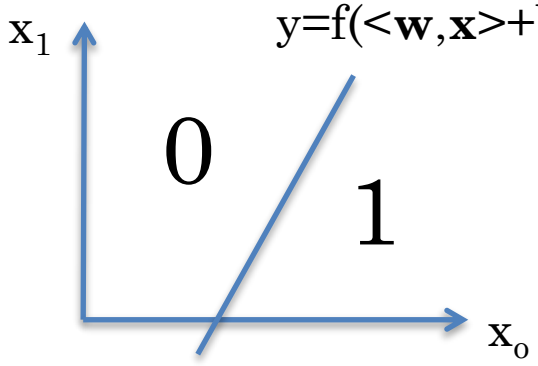
b - смещение

$$f(x) = \begin{cases} 0, x \le 0 \\ 1, x > 0 \end{cases}$$

Пороговая функция

Разделяющая поверхность одного нейрона линейна

$$y = f(z) = f(w_0 x_0 + w_1 x_1 + w_2 x_2 + b) =$$
$$= f(< \vec{w}, \vec{x} > + b)$$



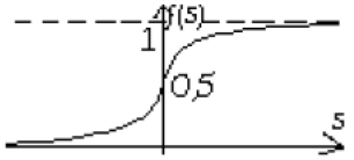
Пороговые функции

$$f(x) = \begin{cases} 0, x \leq 0 \\ 1, x > 0 \end{cases}$$

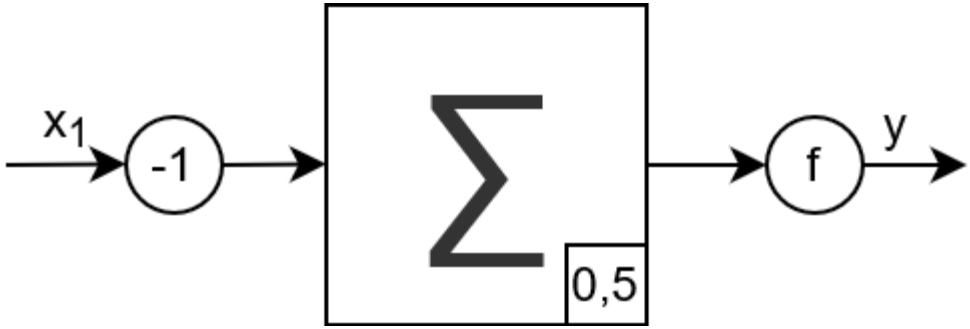
Пороговая функция

$$f(S) = \frac{1}{1 + e^{-\alpha S}}$$

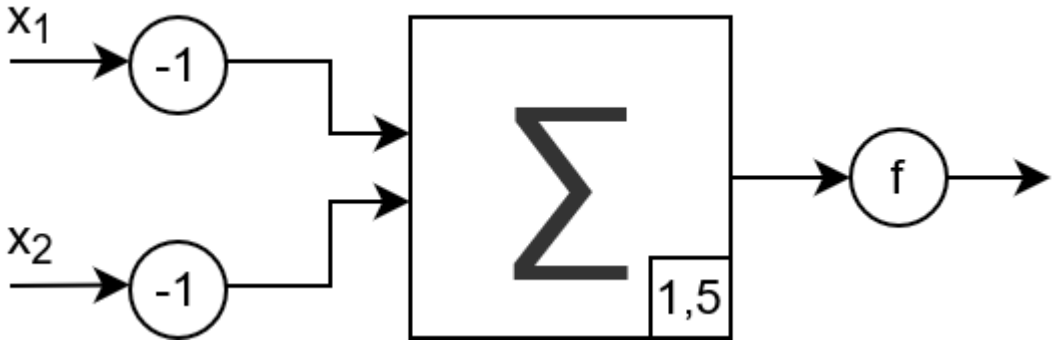
Сигмоида



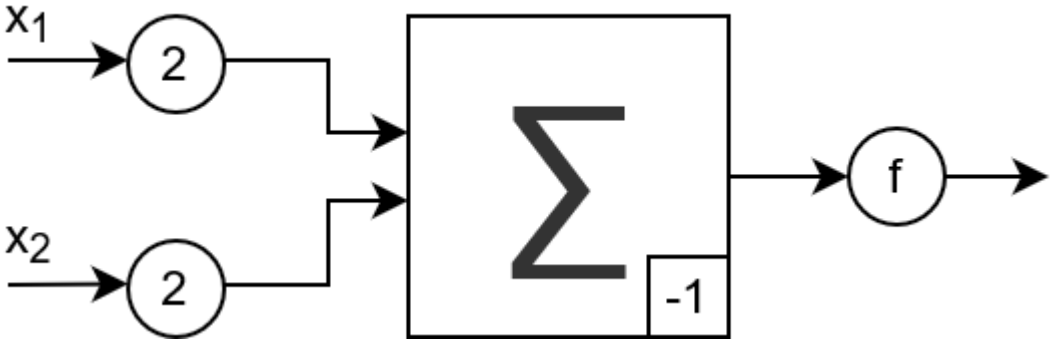
Инициализация двоичных функций



Отрицание

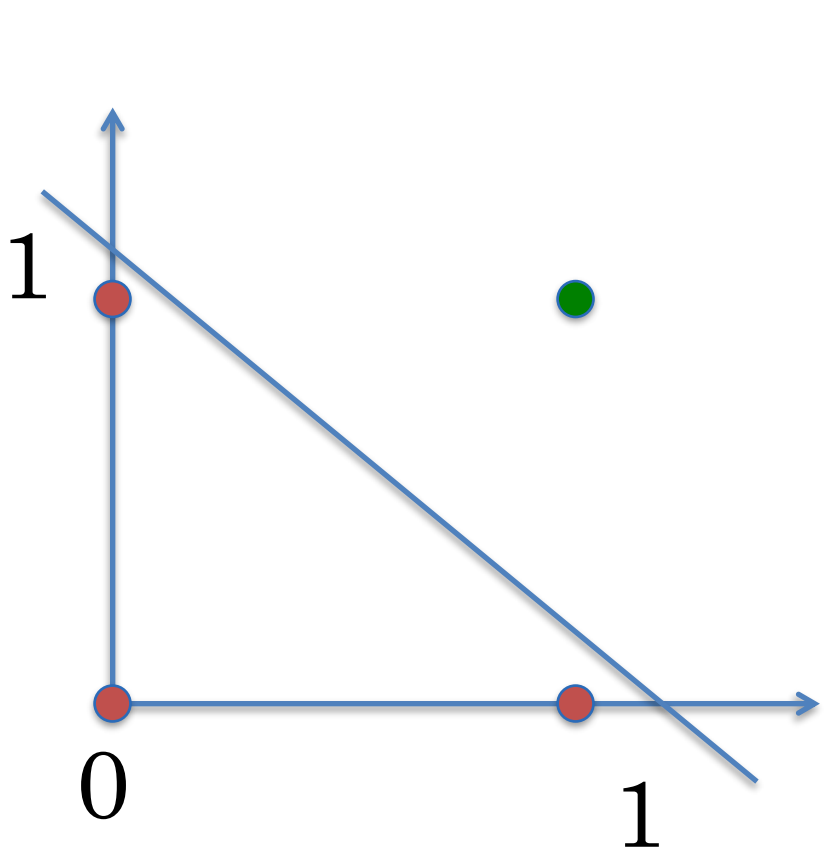


Конъюнкция



Дизъюнкция

Разделяющая поверхность для конъюнкции

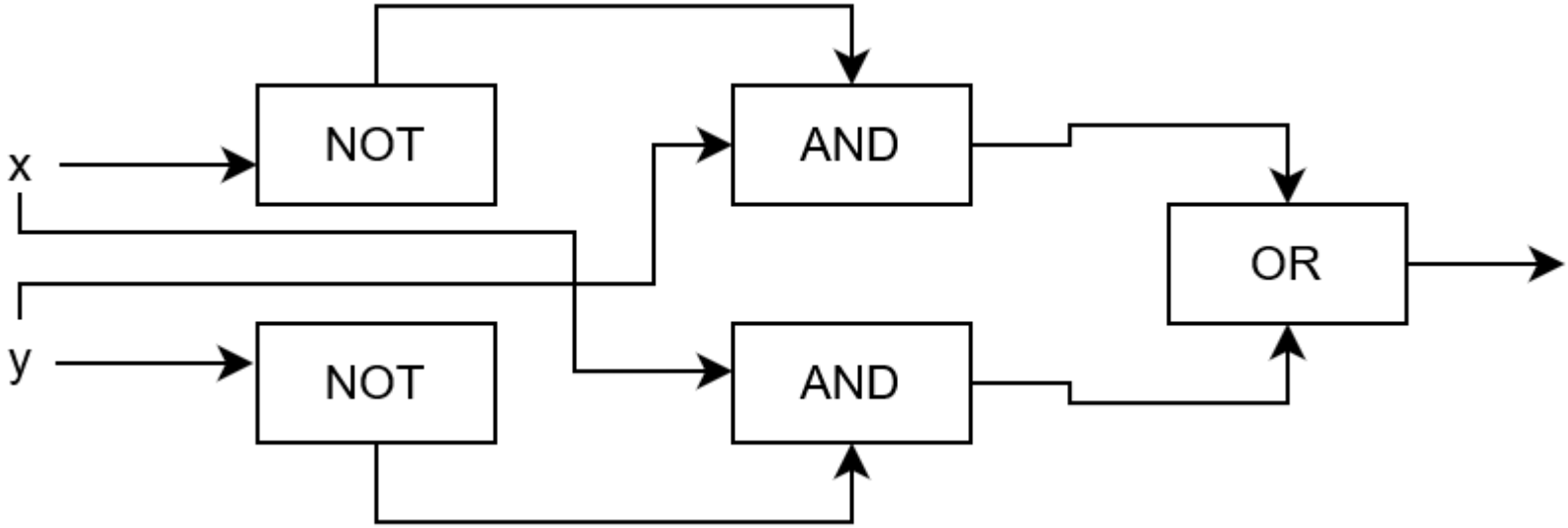


0

1

$$\begin{cases} 1w_1 + 1w_2 + b > 0 \\ 1w_1 + 0w_2 + b \leq 0 \\ 0w_1 + 1w_2 + b \leq 0 \\ 0w_1 + 0w_2 + b \leq 0 \end{cases}$$

«Исключающее или» (XOR)



$$\begin{cases} 1w_1 + 1w_2 + b \leq 0 \\ 1w_1 + 0w_2 + b > 0 \\ 0w_1 + 1w_2 + b > 0 \\ 0w_1 + 0w_2 + b \leq 0 \end{cases}$$

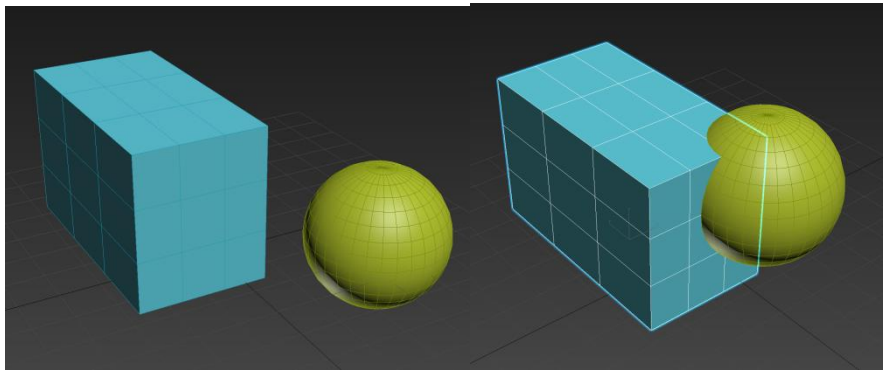
В итоге…

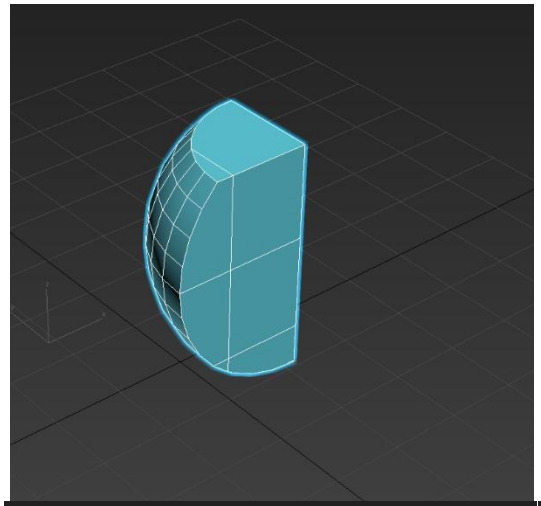
- Важно отметить, что феномен человеческого поведения, есть следствие функционирования биологической структуры. Следовательно, чтобы имитировать разумное поведение, необходимо хорошо понимать принципы и особенности функционирования биологических структур. К сожалению, в науке биологии пока не представлен четкий алгоритм: как работает нейрон, как понимает, куда необходимо отращивать свои дендриты, как настроить свои синапсы, чтобы в нервной системе смог сформироваться простой условный рефлекс…
- С другой стороны в науке об искусственном интеллекте, в восходящем (биологическом) подходе, сложилось парадоксальная ситуация, а именно: когда используемые в исследованиях модели основаны на устаревших представлениях о биологическом нейроне, консерватизм, в основе которого берётся персептрон без переосмысления его основных принципов, без обращения к биологическому первоисточнику, придумывается все более хитроумные алгоритмы и структуры, не имеющих биологических корней. Классические нейронные сети дали множество полезных программных продуктов, но игра с ними не является путем к созданию интеллектуально действующей системы.
- Не редки заявления, о том, что нейрон подобен мощной вычислительной машине, приписывают свойство квантовых компьютеров. Из-за этой сверхсложности, нервной системе приписывается невозможность её повторения. Однако, в реальности природа идет по пути простоты и элегантности своих решений, перемещение зарядов на мембране клетки может служить, как для передачи нервного возбуждения, так и для трансляции информации о том, где происходит данная передача.

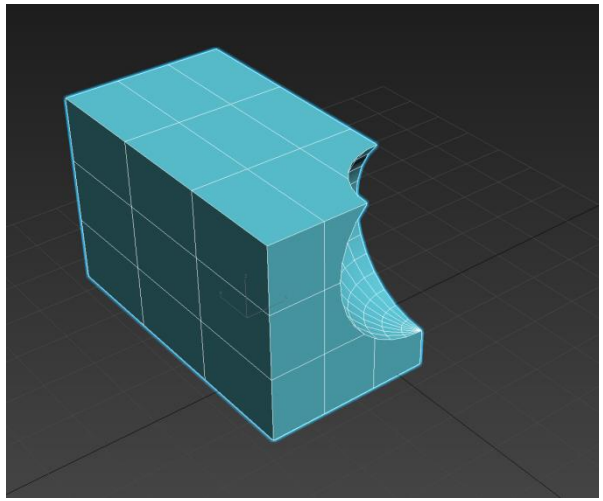
Применение двоичных функций в компьютерной графике

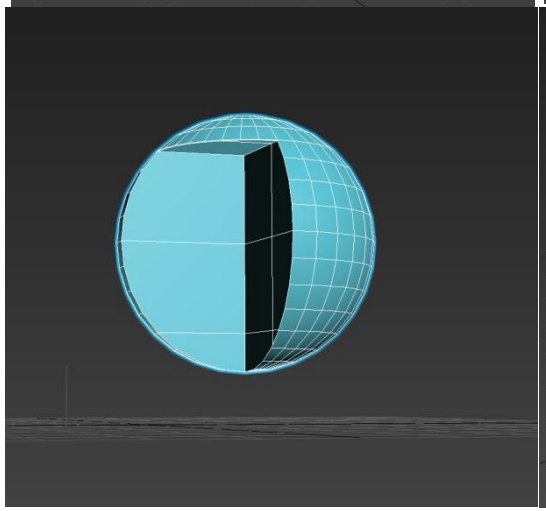


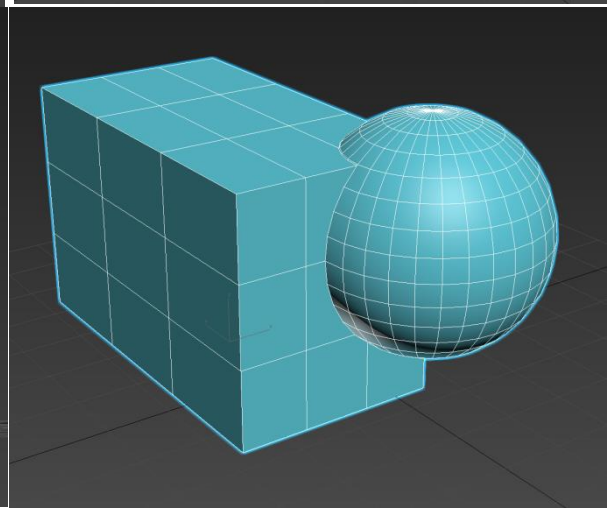
3D-MAX



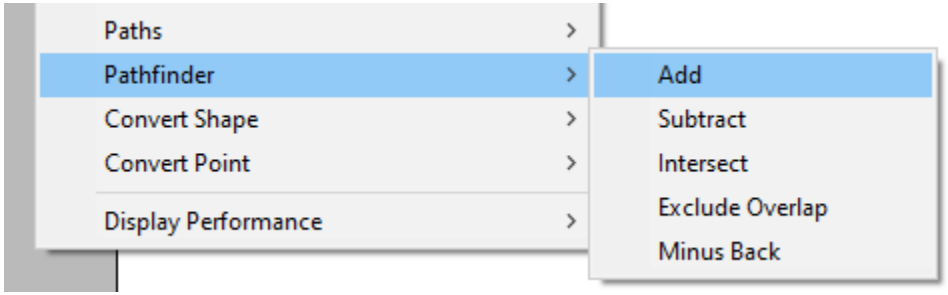


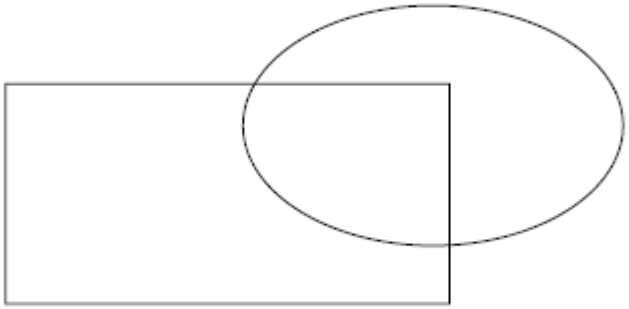


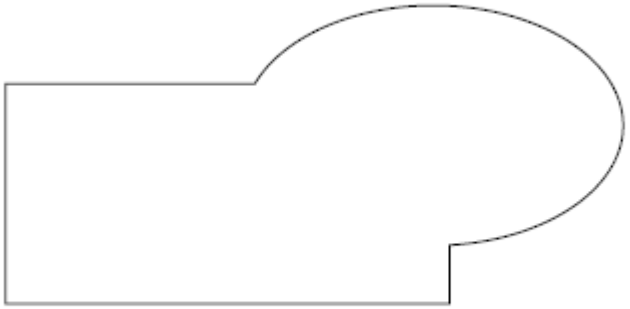


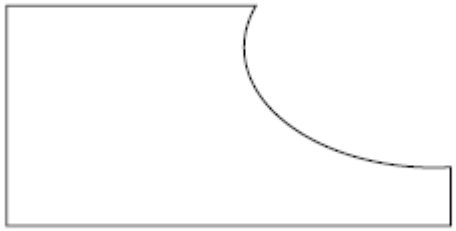


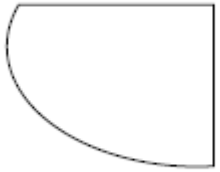
InDesign


Paths
Pathfinder
Convert Shape
Convert Point
Display Performance
Add
Subtract
Intersect
Exclude Overlap
Minus Back









Применение двоичных функций в системах искусственного интеллекта

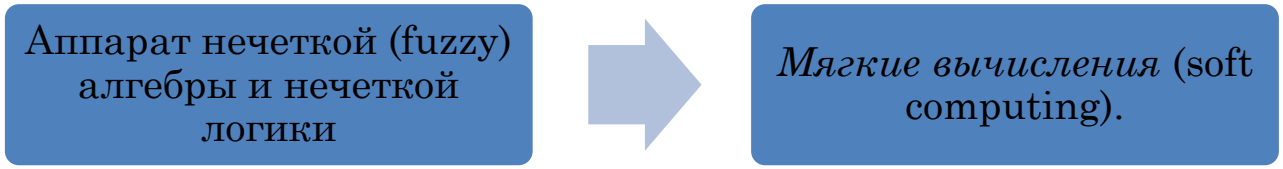


Формальные логические модели

Пример.

- 1)высказывание: a>b представляется термом p(a,b) (двуместный предикат), где p – предикатный символ, заменяющий знак “>";
- 2)высказывание “аппаратная ax – исправна” представляется Q(x);
- 3)теорема Пифагора может быть представлена термом: $P_!$[P_2 (P_3(x),P_3 (y)),P_3(z)]

Нечеткие знания



- **Лингвистическая переменная (ЛП)** — *это переменная, значение которой определяется набором вербальных (то есть словесных) характеристик некоторого свойства.*

Основы теории нечетких множеств

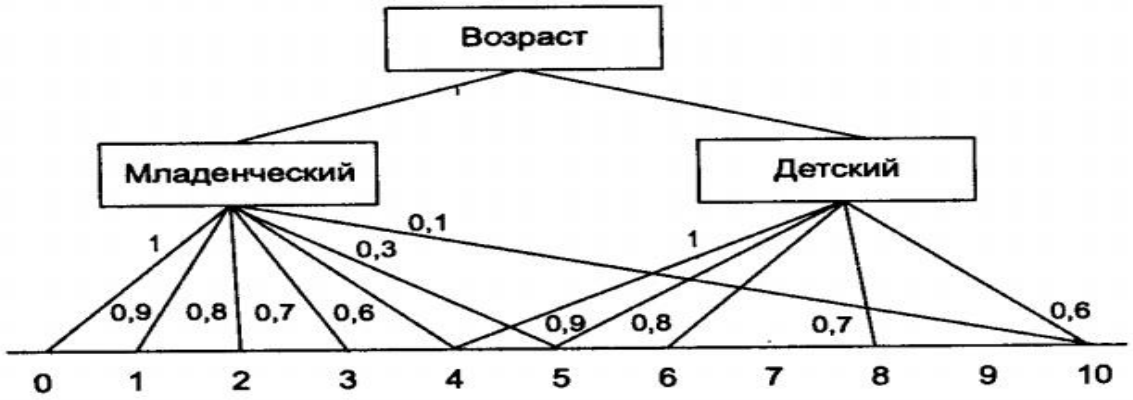


- Нечеткое множество определяется через некоторую базовую шкалу B и функцию принадлежности НМ — μ(x), x ϵ B, ϵ [0...1]. Таким образом, нечеткое множество B — это совокупность пар вида (x, μ(x)), где x ϵ B. где x_i — i-е значение базовой шкалы.
- Функция принадлежности определяет субъективную *степень уверенности* эксперта в том, что данное конкретное значение базовой шкалы соответствует определяемому НМ.
- Эту функцию не стоит путать с вероятностью.

Пример нечетких знаний

Для ЛП «возраст» базовая шкала — это числовая шкала [0,120], обозначающая количество прожитых лет.





Операции с нечеткими знаниями

Операция «ИЛИ» :

- $\mu(x)=max(\mu 1(x), \mu 2(x))$ (логика Заде)
- или так: $\mu(x)=\mu 1(x)+\mu 2(x)- \mu 1(x)*\mu 2(x)$ (вероятностный подход).

Усиление или ослабление лингвистических понятий достигается введением специальных квантификаторов.

$$con(A) = A^2 = \sum_{i=1}^{n} \frac{x_i}{\mu_i^2}$$

СПАСИБО ЗА ВНИМАНИЕ

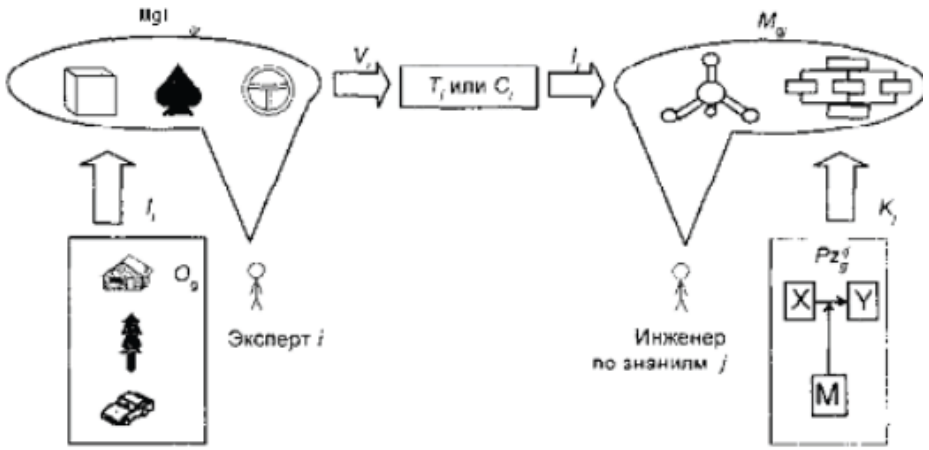
Понятие онтологии

Онтологией называется эксплицитная спецификация концептуализации. Формально онтология состоит из терминов, организованных в таксономию, их определений и атрибутов, а также связанных с ними аксиом и правил вывода.

О*нтология* предметной области включает упорядоченные понятия предметной области (ПО) A и моделирует основные функциональные связи R_A или отношения между понятиями, образующими *Sk*.

Помимо онтологии понимание задачи отражает модель или стратегия принятия решения *Sf* в выбранной ПО.

Схема, отображающая отношения между реальной действительностью и полем знаний:



Формальная модель онтологии

Под *формальной моделью онтологии* **O** будем понимать упорядоченную тройку вида:

$$O =< X, \Re, \Phi >,$$

где

X - конечное множество концептов (понятий, терминов) предметной области, которую представляет онтология **O**;
$\Re$ - конечное множество отношений между концептами (понятиями, терминами) заданной предметной области;
Ф - конечное множество функций интерпретации (аксиоматизация), заданных на концептах и/или отношениях онтологии **O**.

Под *таксономической структурой* будем понимать иерархическую систему понятий, связанных между собой отношением **is_a** («быть элементом класса»).

Формальная модель онтологической системы

Под *формальной моделью онтологической системы* $\sum^{o}$ будем понимать триплет вида:

$$\sum^{o} =< O^{meta}, \{O^{d\&t}\}, \Xi^{\inf} >,$$

Где O^{meta} – онтология верхнего уровня (метаонтология);

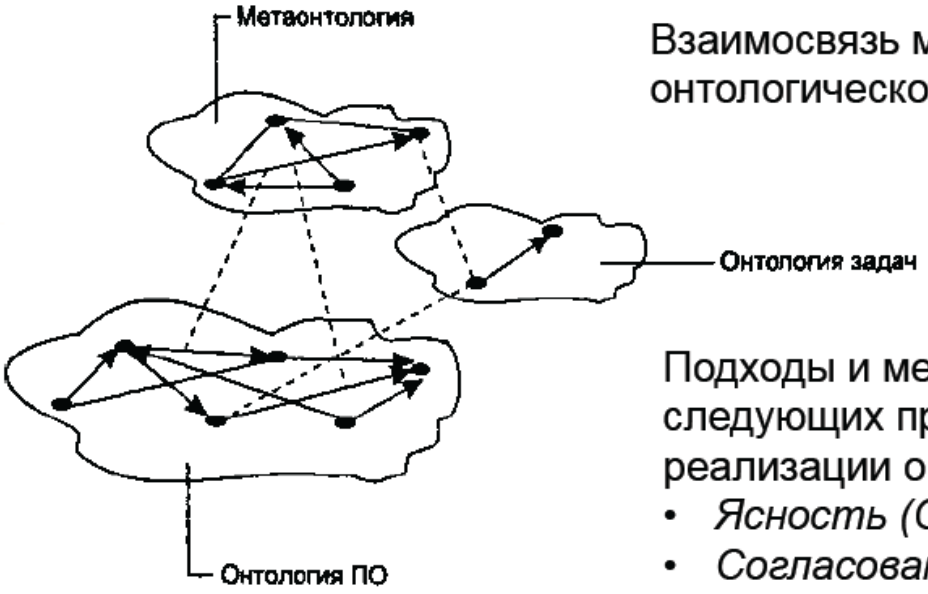
$\{O^{d\&t}\}$ – множество предметных онтологий и онтологий задач предметной области;

$\Xi^{\inf^{f}}$ - модель машины вывода, ассоциированной с онтологической системой $\sum^{o}$.

В модели $\sum^{o}$ имеются три онтологические компоненты:

- метаонтология;
- предметная онтология;
- онтология задач.

Онтологическая система



Взаимосвязь между онтологиями онтологической системы

Подходы и методологии базируются на следующих принципах проектирования и реализации онтологий:

- *Ясность (Clarity)*
- *Согласованность (Coherence)*
- *Расширяемость (Extendibility)*
- *Минимум влияния кодирования (Minimal encoding bias)*
- *Минимум онтологических обязательств (Minimal ontological commitment)*

Онтологический анализ

Статическая онтология
в нее входят сущности предметной области, их свойства и отношения;

Динамическая онтология
определяет состояния, возникающие в процессе решения проблемы, и способ преобразования одних состояний в другие;

Эпистемическая онтология
описывает знания, управляющие процессом перехода из одного состояния в другое.

Онтологический анализ предполагает, что решаемая проблема может быть сведена к проблеме поиска, но при этом не рассматривается, каким именно способом нужно выполнять поиск.

